Experiment 6

Decision Tree

Aim:

To implement the Decision Tree algorithm for classification and analyze its performance.

Theory:

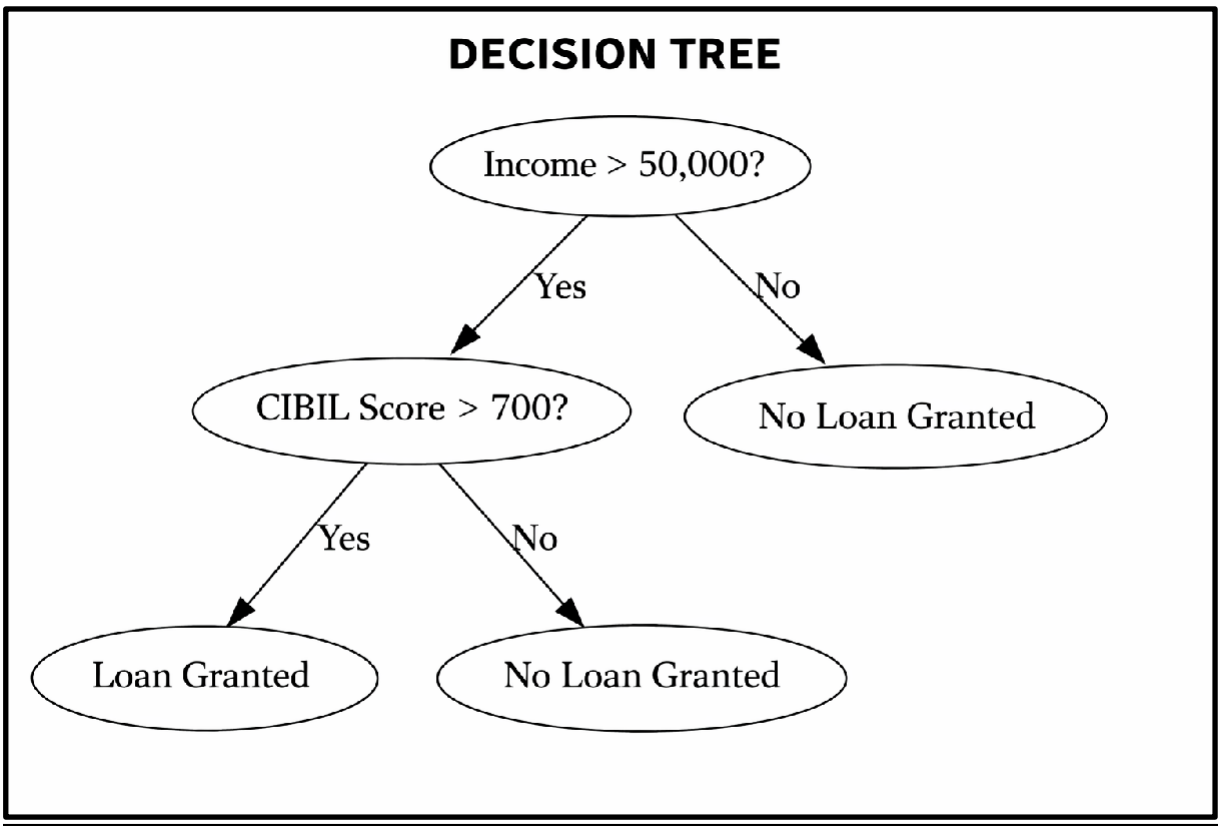
A Decision Tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure consisting of a root node, internal nodes, branches, and leaf nodes. The main components of a Decision Tree are:

- Internal Nodes: Represent attribute tests
- Branches: Represent outcomes of attribute tests.
- Leaf Nodes: Represent final decision or class labels.

As shown in the below figure, a Decision Tree works by recursively subdividing the dataset into smaller groups based on feature values. At each step, the algorithm selects the best feature that separates the data into different classes using an appropriate splitting criterion. This splitting process continues until the data within each node becomes highly uniform or a predefined stopping condition is met. Internal nodes represent decision conditions used to split the data, while leaf nodes represent the final predicted class.

To determine the best split at each node, Decision Trees commonly use Gini Index and Entropy as splitting criteria. The Gini Index measures the impurity of a node and represents the probability of incorrectly classifying a randomly chosen sample if it were labeled according to the class distribution in that node. Lower Gini values indicate purer nodes, and the Gini value becomes zero when all samples belong to a single class. However, it is slightly biased toward dominant classes.

Entropy, on the other hand, measures the amount of uncertainty or disorder in a node and is derived from information theory. It is used to calculate Information Gain, which helps in selecting the most informative feature for splitting. Entropy is zero for a perfectly pure node, while higher entropy values indicate greater disorder. It is sensitive to small changes in class distribution and often produces balanced and informative splits.



Merits of Decision Tree

1. **Easy to Understand and Interpret** – The tree structure is simple and intuitive, making it easy to visualize and explain decisions.

2. **Handles Both Numerical and Categorical Data** – Decision Trees can work with different types of data without much preprocessing.

3. **No Need for Feature Scaling** – Normalization or standardization is not required, as trees are not affected by feature scaling.

Demerits of Decision Tree

1. **Prone to Overfitting** – A deep decision tree may fit noise in the training data, reducing generalization.

2. **Unstable Model** – Small changes in data can lead to a completely different tree structure.

3. **Lower Accuracy Compared to Ensembles** – Single decision trees often perform worse than ensemble methods like Random Forest.

INPUT: Import Libraries

```
# Import necessary libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

from sklearn.preprocessing import OneHotEncoder

from sklearn.compose import ColumnTransformer

import matplotlib.pyplot as plt

import seaborn as sns

print("Libraries imported successfully!")
```

OUTPUT:

```
Libraries imported successfully!
```

INPUT: Load Dataset

```
# Load the dataset

data = pd.read_csv("/content/loan-approval-prediction-
dataset/loan_approval_dataset.csv")

print("Dataset loaded successfully")
```

OUTPUT:

Dataset loaded successfully!

INPUT: Check the shape of the dataset

```
# Check the shape of the dataset

print("Dataset Shape:", data.shape)
```

OUTPUT:

Dataset Shape: (4269, 13)

INPUT: Print columns

```
# Fix column spacing

data.columns = data.columns.str.strip()

# Print columns in vertical form

print("\nColumns (Vertical List):")

for col in data.columns:

    print(col)
```

OUTPUT:

Columns (Vertical List):

loan_id

no_of_dependents

Education

self_employed

Income_annum

loan_amount

loan_term

cibil_score

residential_assets_value

commercial_assets_value

luxury_assets_value

bank_asset_value loan_status

INPUT: Display first 5 rows

Display first 5 rows

print("First 5 rows of the dataset:")

data.head()

OUTPUT:

First 5 rows of the dataset:

First 5 rows of the dataset:

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value	loan_status
0	1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000	Approved
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	Rejected
2	3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000	Rejected
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	Rejected
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000	Rejected

INPUT: Encode Categorical Variables

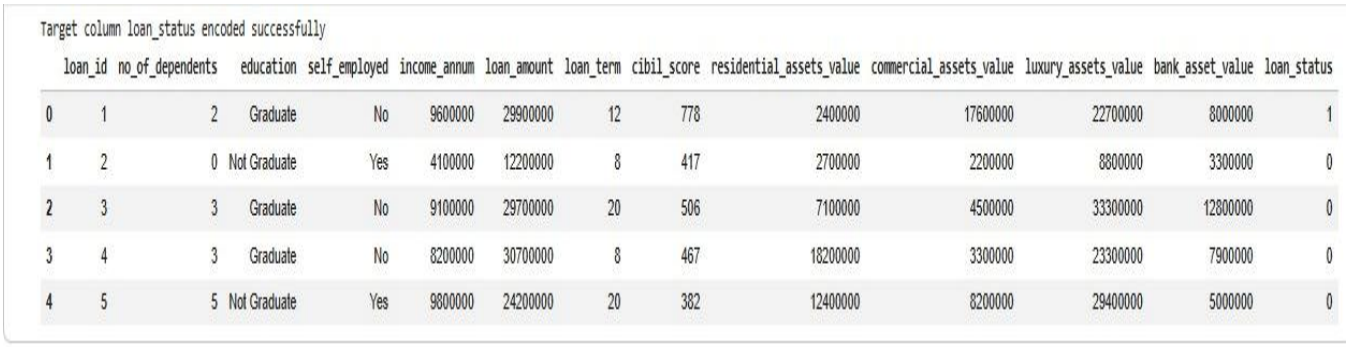
Target Column Encoding

data.columns = data.columns.str.strip()

```
data['loan_status'] = data['loan_status'].str.strip().map({
    'Approved': 1,
    'Rejected': 0
})
print("Target column loan_status encoded successfully")
data.head()
```

OUPTUT:

Target column loan_status encoded successfully



Target column loan_status encoded successfully

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value	loan_status
0	1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000	1
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	0
2	3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000	0
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	0
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000	0

INPUT: Data Information

```
#Data Information
print("All Required Information Related To Data: ")
data.info()
```

OUTPUT:

Data columns (total 13 columns):

Column Non-Null Count Dtype

0 loan_id 4269 non-null int64

1 no_of_dependents 4269 non-null int64

2 education 4269 non-null object

3 self_employed 4269 non-null object

4 income_annum 4269 non-null int64

5 loan_amount 4269 non-null int64

6 loan_term 4269 non-null int64

7 cibil_score 4269 non-null int64

8 residential_assets_value 4269 non-null int64

9 commercial_assets_value 4269 non-null int64

10 luxury_assets_value 4269 non-null int64

11 bank_asset_value 4269 non-null int64

12 loan_status 4269 non-null int64

INPUT: Check for missing (null) values

```
# Check for missing (null) values in each column

print("Missing values in each column:")

print(data.isnull().sum())
```

OUTPUT:

Missing values in each column:

loan_id 0

no_of_dependents 0

education 0

self_employed 0

income_annum 0

loan_amount 0

loan_term 0

cibil_score 0

residential_assets_value 0

commercial_assets_value 0

luxury_assets_value 0

bank_asset_value 0

loan_status 0

dtype: int64

INPUT: Data Transformation

```
# Define target column

target_col = "loan_status"

print("Target column selected:", target_col)

# Separate features and target

X = data.drop(columns=[target_col])

y = data[target_col]
```

```
print("Features (X) and target (y) separated successfully")

# Identify categorical columns

cat_cols = X.select_dtypes(include=['object']).columns.tolist()

print("Categorical columns identified:", cat_cols)

# Apply One-Hot Encoding to categorical columns

from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(

    transformers=[

        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)

    ],

    remainder='passthrough'

)

# Transform feature data

X_transformed = preprocessor.fit_transform(X)

print("Categorical features one-hot encoded and dataset transformed successfully")
```

OUTPUT:

```
Target column selected: loan_status

Features (X) and target (y) separated successfully

Categorical columns identified: ['education', 'self_employed']
```

Categorical features one-hot encoded and dataset transformed successfully

INPUT: Split the Data

```
# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split( X_transformed, y, test_size=0.2,
random_state=42, stratify=y)

print("Dataset successfully split into training and testing sets")

print("Training set size:", X_train.shape)

print("Testing set size:", X_test.shape)
```

OUTPUT

Dataset successfully split into training and testing sets

Training set size: (3415, 14)

Testing set size: (854, 14)

GINI

INPUT: Initialize and train Decision Tree model

```
# Initialize and train Decision Tree model using Gini impurity

dt_model = DecisionTreeClassifier(criterion='gini', random_state=42)

dt_model.fit(X_train, y_train)

print("Decision Tree model trained successfully using Gini impurity")
```

OUTPUT:

Decision Tree model trained successfully using Gini impurity

INPUT: Evaluate Decision Tree model

```
# Evaluate Decision Tree model (Gini impurity)

y_pred = dt_model.predict(X_test)

print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix (Gini, Full Tree):")

print(confusion_matrix(y_test, y_pred))
```

OUTPUT:

```
Accuracy (Gini, Full Tree): 0.9789227166276346

Confusion Matrix (Gini, Full Tree):

[[313 10]

[ 8 523]]
```

INPUT: Compute confusion matrix

```
# Compute confusion matrix for Decision Tree model (Gini impurity)

print("Calculating confusion matrix (Gini)...")

cm = confusion_matrix(y_test, y_pred)

print("Confusion matrix calculated successfully")
```

OUTPUT:

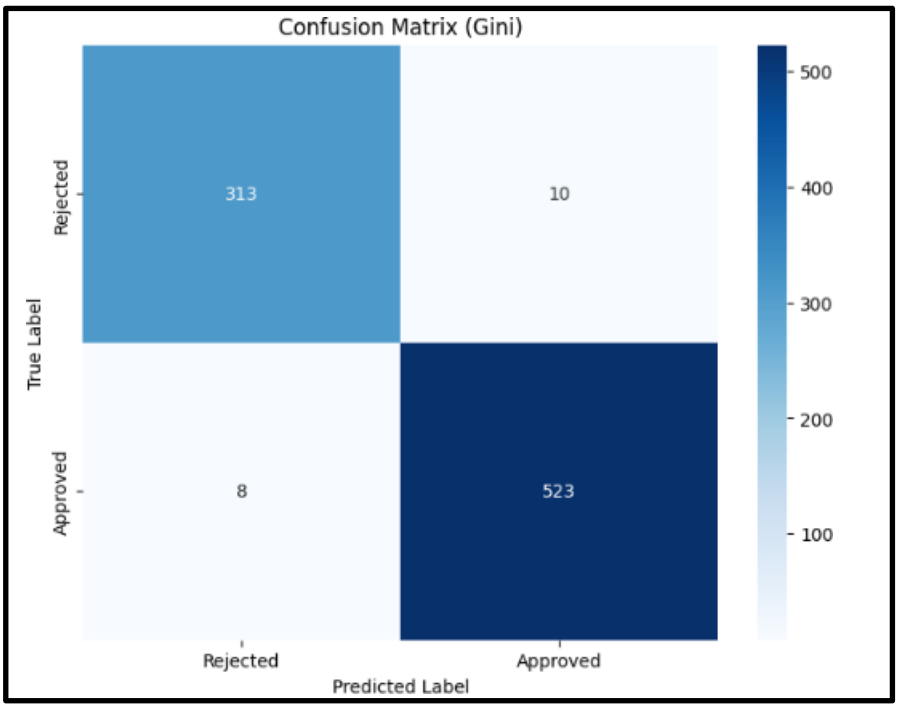
```
Confusion matrix calculated successfully
```

INPUT: Plot confusion matrix

```
# Plot confusion matrix for Decision Tree model (Gini impurity)
```

```
print("Displaying confusion matrix plot (Gini)...")

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',  xticklabels=target_names,
yticklabels=target_names)

plt.title('Confusion Matrix (Gini)')

plt.ylabel('True Label')

plt.xlabel('Predicted Label')

plt.show()

print("Confusion matrix plot displayed successfully")
```

OUTPUT:



ENTROPY

INPUT: Initialize and train Decision Tree model

```
# Train and evaluate Decision Tree model using Entropy

dt_model_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)

dt_model_entropy.fit(X_train, y_train)

print("Decision Tree model trained successfully using Entropy")

# Predict on test data

y_pred_entropy = dt_model_entropy.predict(X_test)

print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))

print("\nConfusion Matrix (Entropy, Full Tree):")

print(confusion_matrix(y_test, y_pred_entropy))
```

OUTPUT

```
Decision Tree model trained successfully using Entropy

Accuracy (Entropy, Full Tree): 0.9789227166276346

Confusion Matrix (Entropy, Full Tree):

[[312 11]

[ 7 524]]
```

INPUT: Compute confusion matrix

```
# Compute confusion matrix for Decision Tree model (Entropy)

print("Calculating confusion matrix (Entropy)...")

cm_entropy = confusion_matrix(y_test, y_pred_entropy)

print("Confusion matrix (Entropy) calculated successfully")
```

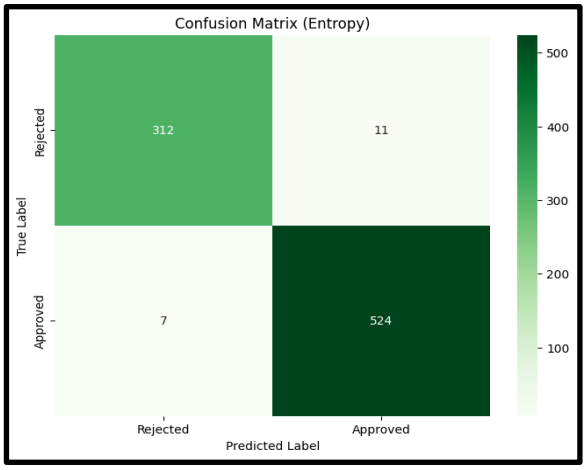
OUTPUT:

Confusion matrix (Entropy) calculated successfully

INPUT: Plot confusion matrix

```
# Plot confusion matrix for Decision Tree model (Entropy)

# Define target class names

target_names = ['Rejected', 'Approved']

plt.figure(figsize=(8, 6))

sns.heatmap(  cm_entropy, annot=True, fmt='d', cmap='Greens',   xticklabels=target_names,

    yticklabels=target_names)

plt.title('Confusion Matrix (Entropy)')

plt.ylabel('True Label')

plt.xlabel('Predicted Label')

plt.show()

print("Confusion matrix (Entropy) plot displayed successfully")
```

OUTPUT:



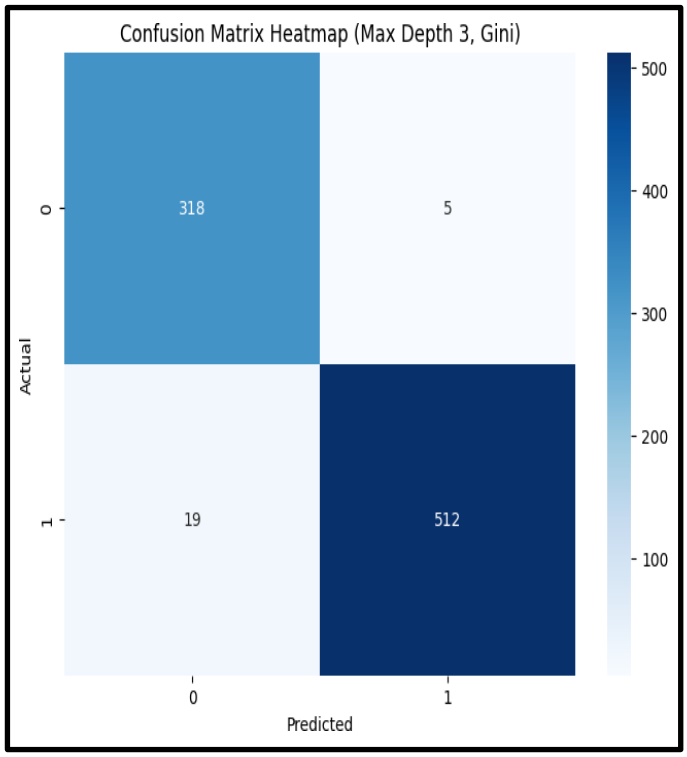
INPUT: GINI(MAX DEPTH: 3)

```
# Train Decision Tree with max_depth=3

dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)

dt_gini_3.fit(X_train, y_train)

# Predictions

y_pred_3 = dt_gini_3.predict(X_test)

# Accuracy

accuracy_3 = accuracy_score(y_test, y_pred_3)

print(f"Accuracy (Max Depth 3, Gini): {accuracy_3:.4f}")

# Confusion Matrix

cm_3 = confusion_matrix(y_test, y_pred_3)

print("Confusion Matrix (Max Depth 3):")

print(cm_3)

# Heatmap
```

```
plt.figure(figsize=(8, 6))

sns.heatmap(cm_3, annot=True, fmt='d', cmap='Blues', xticklabels=True, yticklabels=True)

plt.title("Confusion Matrix Heatmap (Max Depth 3, Gini)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

OUTPUT:

Accuracy (Max Depth 3, Gini): 0.9719

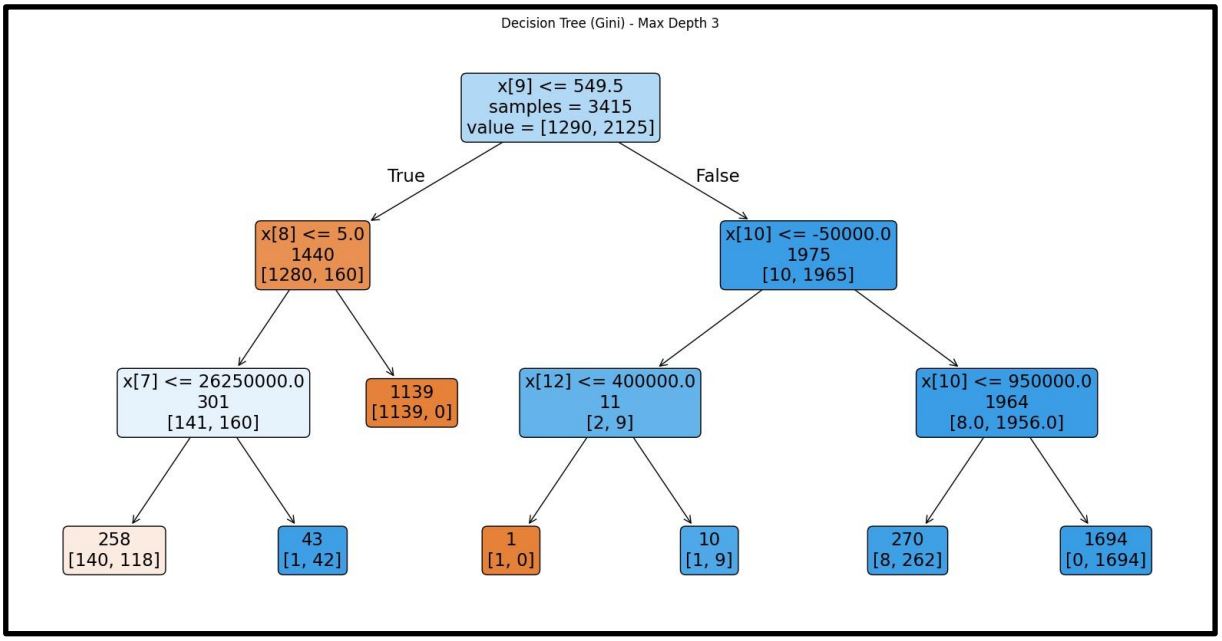


INPUT: Plot Decision Tree

```
# Train Decision Tree with max_depth=3

dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
```

```
dt_gini_3.fit(X_train, y_train)

# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(dt_gini_3, feature_names=None, class_names=None,

    filled=True,

    rounded=True,

    label='root',

    impurity=False,

    proportion=False

)

plt.title("Decision Tree (Gini) - Max Depth 3")

plt.show()
```

OUTPUT:



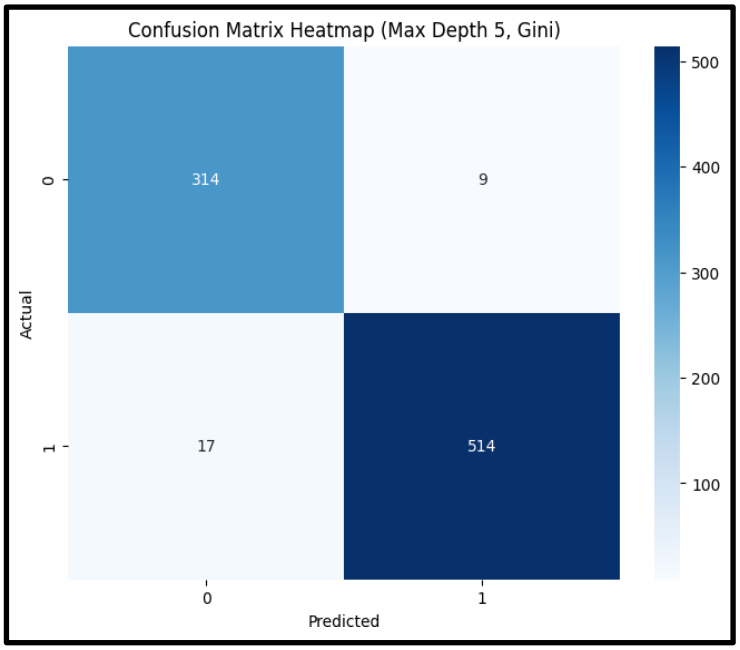
INPUT: GINI(MAX DEPTH: 5)

```
# Train Decision Tree with max_depth=5

dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)

dt_gini_5.fit(X_train, y_train)

# Predictions

y_pred_5 = dt_gini_5.predict(X_test)

# Accuracy

accuracy_5 = accuracy_score(y_test, y_pred_5)

print(f"Accuracy (Max Depth 5, Gini): {accuracy_3:.4f}")

# Confusion Matrix

cm_5 = confusion_matrix(y_test, y_pred_5)

print("Confusion Matrix (Max Depth 5):")
```

```
print(cm_5)

# Heatmap

plt.figure(figsize=(8, 6))

sns.heatmap(cm_5, annot=True, fmt='d', cmap='Blues', xticklabels=True, yticklabels=True)

plt.title("Confusion Matrix Heatmap (Max Depth 5, Gini)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

OUTPUT:

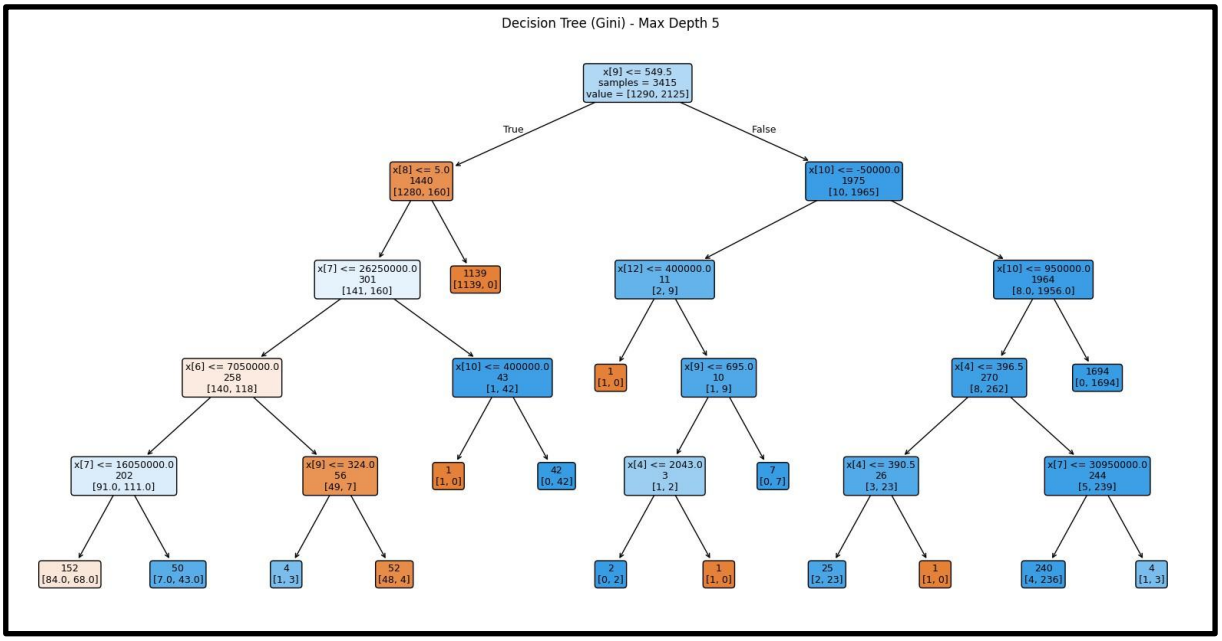
Accuracy (Max Depth 5, Gini): 0.9696



INPUT: Plot Decision Tree

```
# Train Decision Tree with max_depth=5
```

```
dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)

dt_gini_5.fit(X_train, y_train)

# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(

    dt_gini_5,

    feature_names=None,

    class_names=None,

    filled=True,

    rounded=True,

    label='root',

    impurity=False,

    proportion=False

)

plt.title("Decision Tree (Gini) - Max Depth 5")

plt.show()
```



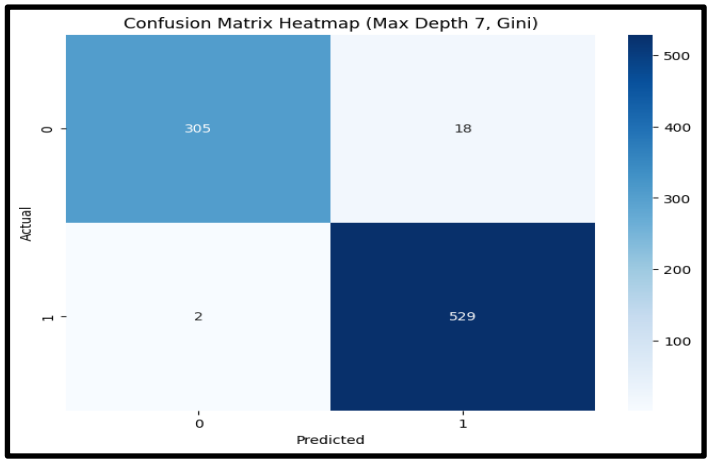
INPUT: GINI(MAX DEPTH: 7)

```
# Train Decision Tree with max_depth=7

dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)

dt_gini_7.fit(X_train, y_train)

# Predictions

y_pred_7 = dt_gini_7.predict(X_test)

# Accuracy

accuracy_7 = accuracy_score(y_test, y_pred_7)

print(f"Accuracy (Max Depth 7, Gini): {accuracy_7:.4f}")

# Confusion Matrix

cm_7 = confusion_matrix(y_test, y_pred_7)

print("Confusion Matrix (Max Depth 7):")
```

```
print(cm_7)

# Heatmap

plt.figure(figsize=(8, 6))

sns.heatmap(cm_7, annot=True, fmt='d', cmap='Blues', xticklabels=True, yticklabels=True)

plt.title("Confusion Matrix Heatmap (Max Depth 7, Gini)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

OUTPUT:

Accuracy (Max Depth 7, Gini): 0.9766

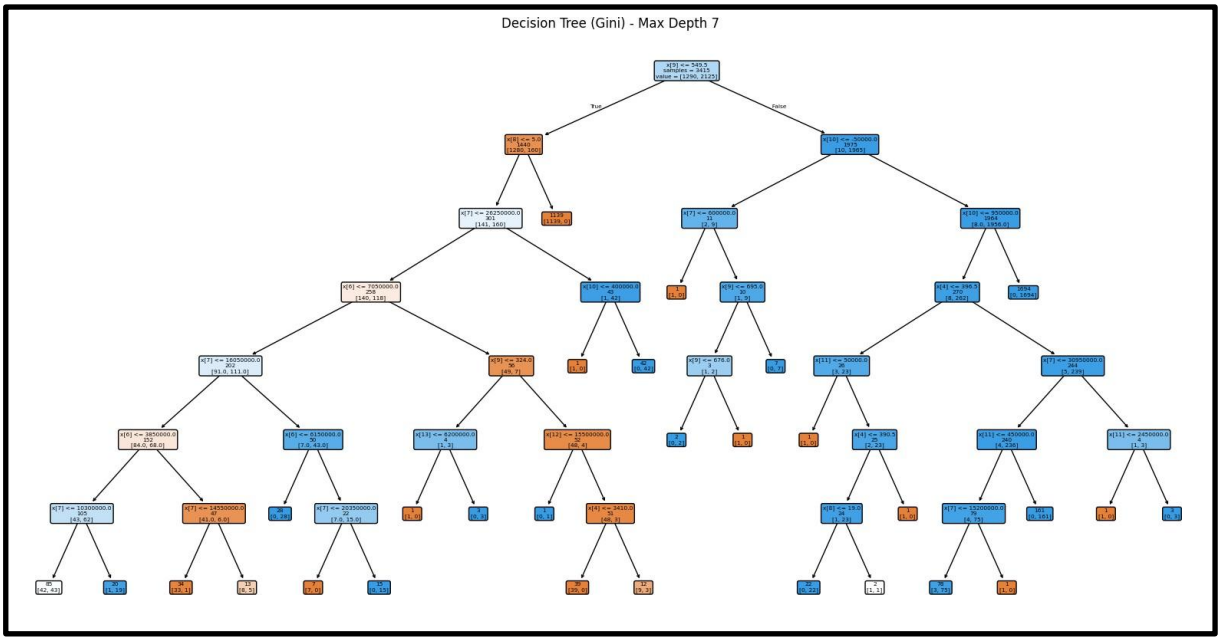


INPUT : Plot The Decision Tree

```
# Train Decision Tree with max_depth=7

dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)

dt_gini_7.fit(X_train, y_train)

# Plot the tree
```

```
plt.figure(figsize=(20, 10))

plot_tree(

    dt_gini_7,

    feature_names=None,

    class_names=None,

    filled=True,

    rounded=True,

    label='root',

    impurity=False,

    proportion=False

)

plt.title("Decision Tree (Gini) - Max Depth 7")

plt.show()
```

OUTPUT:



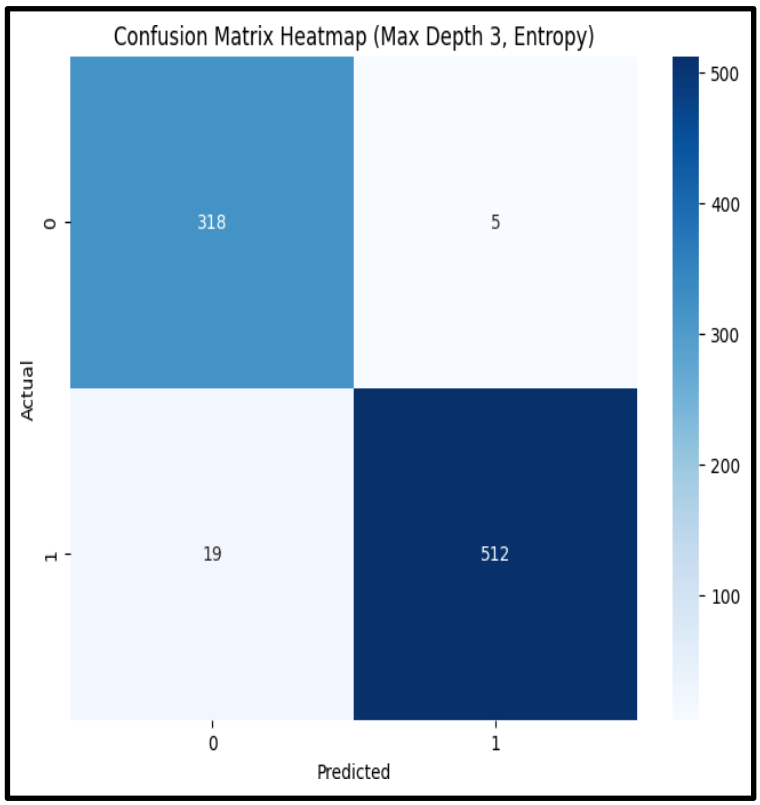
INPUT:ENTROPY (MAX DEPTH:3)

```
# Train Decision Tree with max_depth = 3 (Entropy)
dt_entropy_3 = DecisionTreeClassifier(
criterion='entropy',
max_depth=3,
random_state=42
)
dt_entropy_3.fit(X_train, y_train)
# Plot the tree
plt.figure(figsize=(20, 10))
plot_tree(
dt_entropy_3,
feature_names=None,
class_names=None,
filled=True,
rounded=True,
label='root',
impurity=False,
proportion=False
)
```

plt.title("Decision Tree (Entropy) - Max Depth 3")

plt.show()

OUTPUT:



INPUT : Plot The Decision Tree

Train Decision Tree with max_depth = 3 (Entropy)

dt_entropy_3 = DecisionTreeClassifier(

criterion='entropy',

max_depth=3,

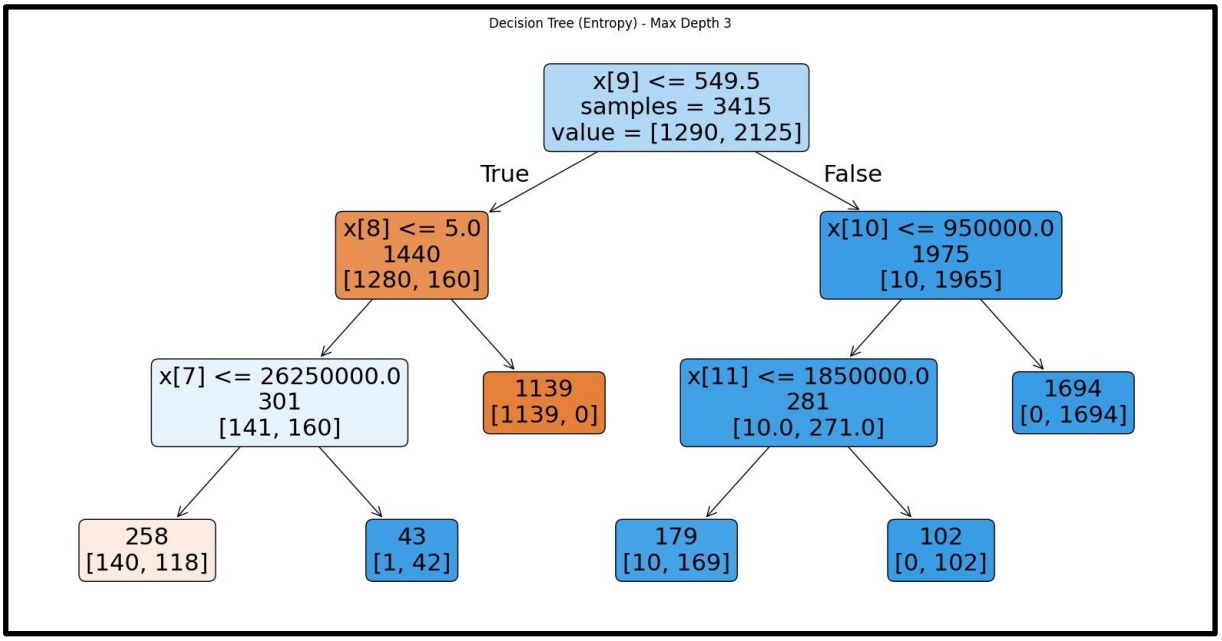
random_state=42

)

dt_entropy_3.fit(X_train, y_train)

```
# Plot the tree
plt.figure(figsize=(20, 10))
plot_tree(
dt_entropy_3,
feature_names=None,
class_names=None,
filled=True,
rounded=True,
label='root',
impurity=False,
proportion=False
)
plt.title("Decision Tree (Entropy) - Max Depth 3")
plt.show()
```

OUTPUT

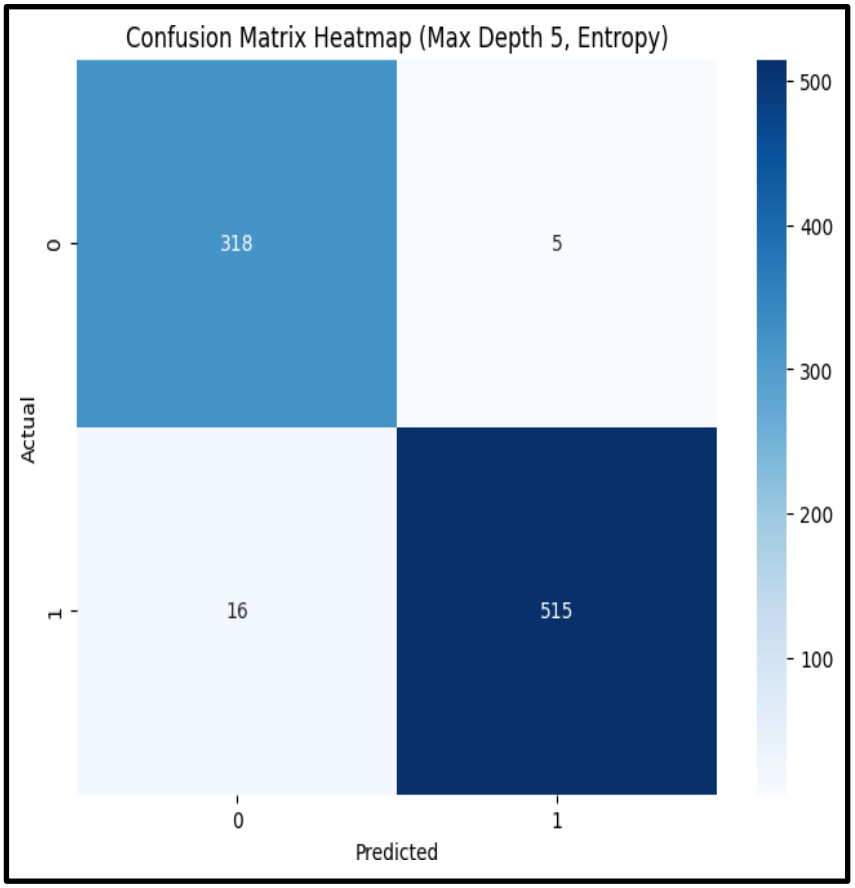


INPUT:ENTROPY (MAX DEPTH:5)

```
# Train Decision Tree with max_depth = 5 (Entropy)

dt_entropy_5 = DecisionTreeClassifier(

criterion='entropy',

max_depth=5,

random_state=42

)

dt_entropy_5.fit(X_train, y_train)

# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(

dt_entropy_5,
```

```
feature_names=None,

class_names=None,

filled=True,

rounded=True,

label='root',

impurity=False,

proportion=False

)

plt.title("Decision Tree (Entropy) - Max Depth 5")

plt.show()
```

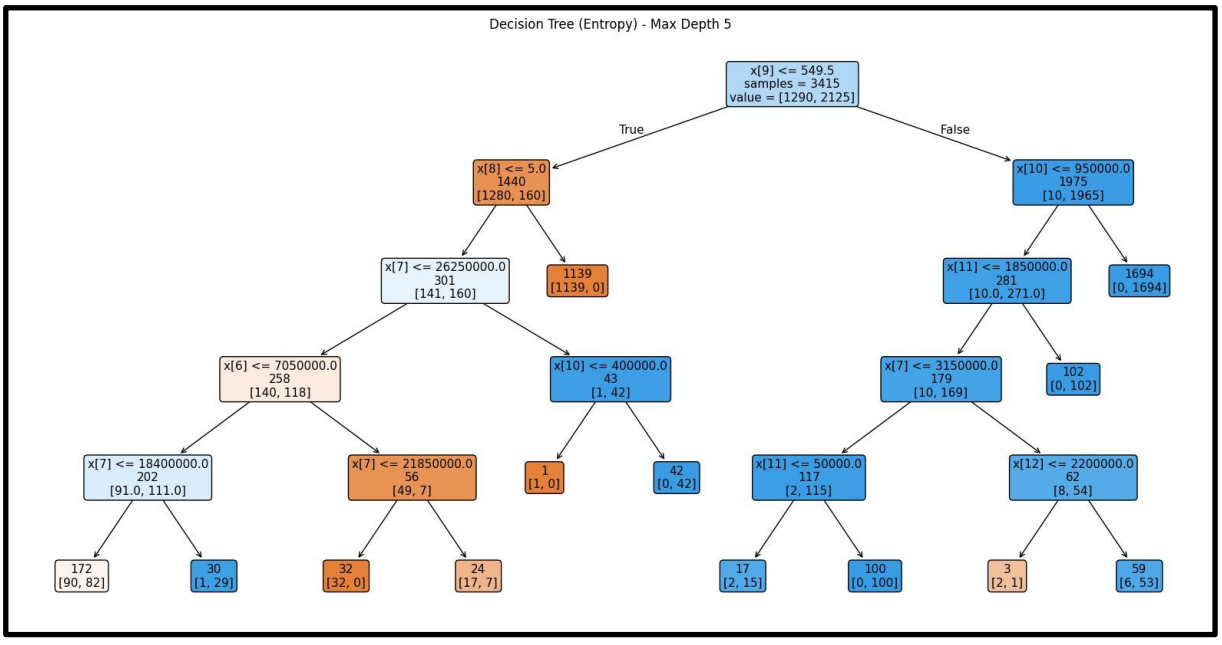
OUTPUT:



INPUT : Plot The Decision Tree

```
# Train Decision Tree with max_depth = 5 (Entropy)

dt_entropy_5 = DecisionTreeClassifier(

criterion='entropy',

max_depth=5,

random_state=42

)

dt_entropy_5.fit(X_train, y_train)

# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(
```

```
dt_entropy_5,

feature_names=None,

class_names=None,

filled=True,

rounded=True,

label='root',

impurity=False,

proportion=False

)

plt.title("Decision Tree (Entropy) - Max Depth 5")

plt.show()
```

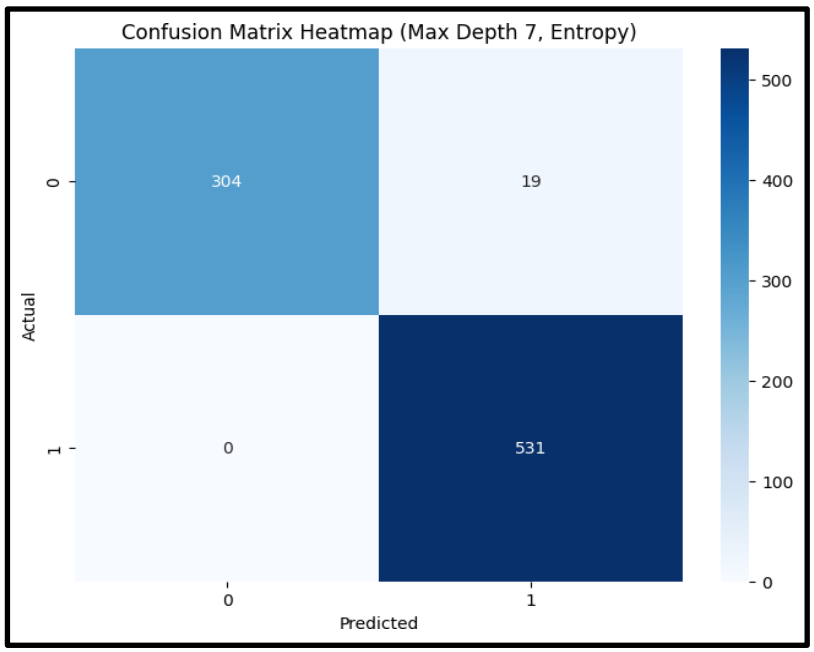


INPUT:ENTROPY (MAX DEPTH:7)

```
# Train Decision Tree with max_depth = 7 (Entropy)
```

```
dt_entropy_7 = DecisionTreeClassifier(
criterion='entropy',
max_depth=7,
random_state=42
)
dt_entropy_7.fit(X_train, y_train)
# Plot the tree
plt.figure(figsize=(20, 10))
plot_tree(
dt_entropy_7,
feature_names=None,
class_names=None,
filled=True,
rounded=True,
label='root',
impurity=False,
proportion=False
)
plt.title("Decision Tree (Entropy) - Max Depth 7")
plt.show()
```

OUTPUT:



INPUT : Plot The Decision Tree

```
# Train Decision Tree with max_depth = 7 (Entropy)

dt_entropy_7 = DecisionTreeClassifier(

criterion='entropy',

max_depth=7,

random_state=42

)

dt_entropy_7.fit(X_train, y_train)

# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(

dt_entropy_7,
```

feature_names=None,

class_names=None,

filled=True,

rounded=True,

label='root',

impurity=False,

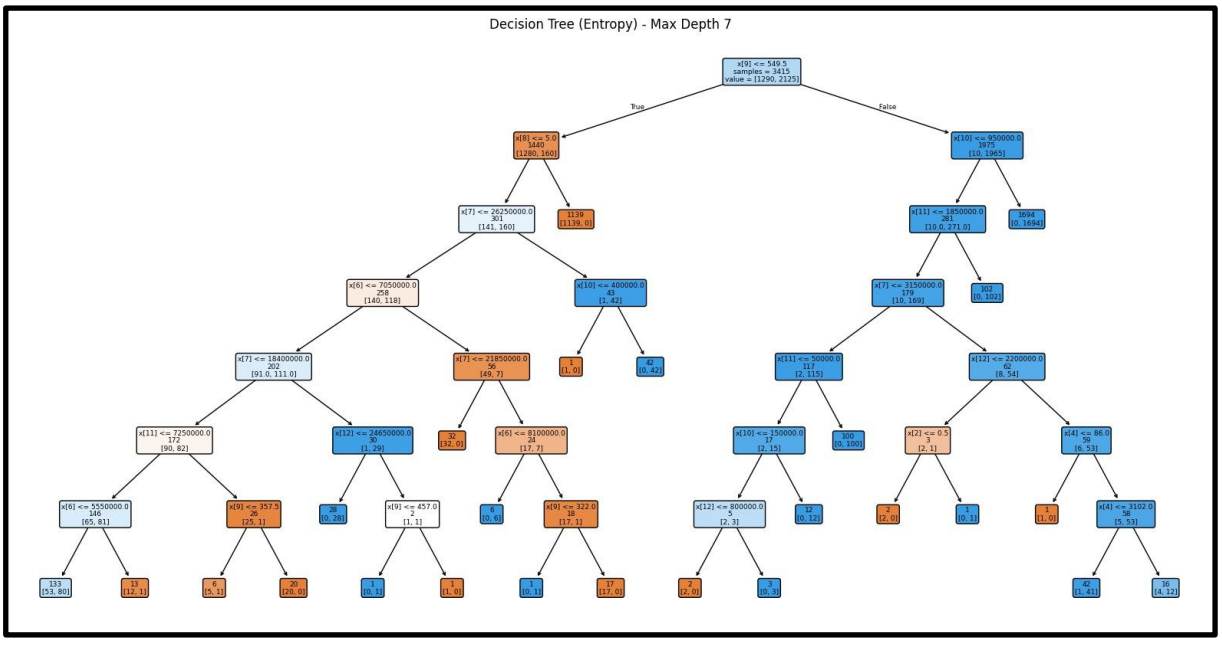
proportion=False

)

plt.title("Decision Tree (Entropy) - Max Depth 7")

plt.show()

OUTPUT



MULTI CLASSIFICATION

INPUT: Import Libraries

Import necessary libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn.preprocessing import LabelEncoder

print("Libraries imported successfully!")
```

OUTPUT:

```
Libraries imported successfully
```

INPUT: Load Dataset

```
# Load the dataset

data = pd.read_csv('ObesityDataSet_raw_and_data_sinthetic.csv')

print("Dataset loaded successfully")
```

OUTPUT:

```
Dataset loaded successfully
```

INPUT: Check Shape of Dataset

```
# Check the shape of the dataset

print("Data Shape:")

print(data.shape)
```

OUTPUT:

Data Shape:(2111, 17)

INPUT: Display first 5 rows

```
# Display first 5 rows

print("First 5 rows of the dataset:")

data.head()
```

OUTPUT:

```
First 5 rows of the dataset:
```

	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP	CAEC	SMOKE	CH2O	SCC	FAF	TUE	CALC	MTRANS	NObeyesdad
0	Female	21.0	1.62	64.0	yes	no	2.0	3.0	Sometimes	no	2.0	no	0.0	1.0	no	Public_Transportation	Normal_Weight
1	Female	21.0	1.52	56.0	yes	no	3.0	3.0	Sometimes	yes	3.0	yes	3.0	0.0	Sometimes	Public_Transportation	Normal_Weight
2	Male	23.0	1.80	77.0	yes	no	2.0	3.0	Sometimes	no	2.0	no	2.0	1.0	Frequently	Public_Transportation	Normal_Weight
3	Male	27.0	1.80	87.0	no	no	3.0	3.0	Sometimes	no	2.0	no	2.0	0.0	Frequently	Walking	Overweight_Level_I
4	Male	22.0	1.78	89.8	no	no	2.0	1.0	Sometimes	no	2.0	no	0.0	0.0	Sometimes	Public_Transportation	Overweight_Level_II

INPUT: Display last 5 rows

```
# Display last 5 rows

print("Last 5 rows of the dataset:")

data.tail()
```

OUTPUT:

```
Last 5 rows of the dataset:
```

	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP	CAEC	SMOKE	CH2O	SCC	FAF	TUE	CALC	MTRANS	NObeyesdad
2106	Female	20.976842	1.710730	131.408528	yes	yes	3.0	3.0	Sometimes	no	1.728139	no	1.676269	0.906247	Sometimes	Public_Transportation	Obesity_Type_III
2107	Female	21.982942	1.748584	133.742943	yes	yes	3.0	3.0	Sometimes	no	2.005130	no	1.341390	0.599270	Sometimes	Public_Transportation	Obesity_Type_III
2108	Female	22.524036	1.752206	133.689352	yes	yes	3.0	3.0	Sometimes	no	2.054193	no	1.414209	0.646288	Sometimes	Public_Transportation	Obesity_Type_III
2109	Female	24.361936	1.739450	133.346641	yes	yes	3.0	3.0	Sometimes	no	2.852339	no	1.139107	0.586035	Sometimes	Public_Transportation	Obesity_Type_III
2110	Female	23.664709	1.738836	133.472641	yes	yes	3.0	3.0	Sometimes	no	2.863513	no	1.026452	0.714137	Sometimes	Public_Transportation	Obesity_Type_III

INPUT: Encode Categorical Variables

```
# Initialize LabelEncoder

le = LabelEncoder()
```

```
target_names = []

print("LabelEncoder initialized")

# Apply encoding to categorical columns

for col in data.columns:

if data[col].dtype == 'object':

data[col] = le.fit_transform(data[col])
```

```
if col == 'NObeyesdad':

target_names = list(le.classes_)

print("Categorical variables encoded")
```

OUTPUT:

```
LabelEncoder initialized

Categorical variables encoded
```

INPUT: Split Data

```
# Split data into training and testing sets

X = data.drop('NObeyesdad', axis=1)

y = data['NObeyesdad']

X_train, X_test, y_train, y_test = train_test_split(X, y,

test_size=0.2,

random_state=42)

print("Data split complete")
```

OUTPUT:

```
Data split complete
```

GINI

INPUT: Train Decision Tree

```
# Initialize and train Decision Tree with Gini impurity

model = DecisionTreeClassifier(criterion='gini')
```

```
model.fit(X_train, y_train)

print("Model trained using Gini impurity")
```

OUTPUT:

```
Model trained using Gini impurity
```

INPUT: Evaluate Model

```
# Evaluate Gini model

y_pred = model.predict(X_test)

print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix (Gini, Full Tree):\n", confusion_matrix(y_test, y_pred))
```

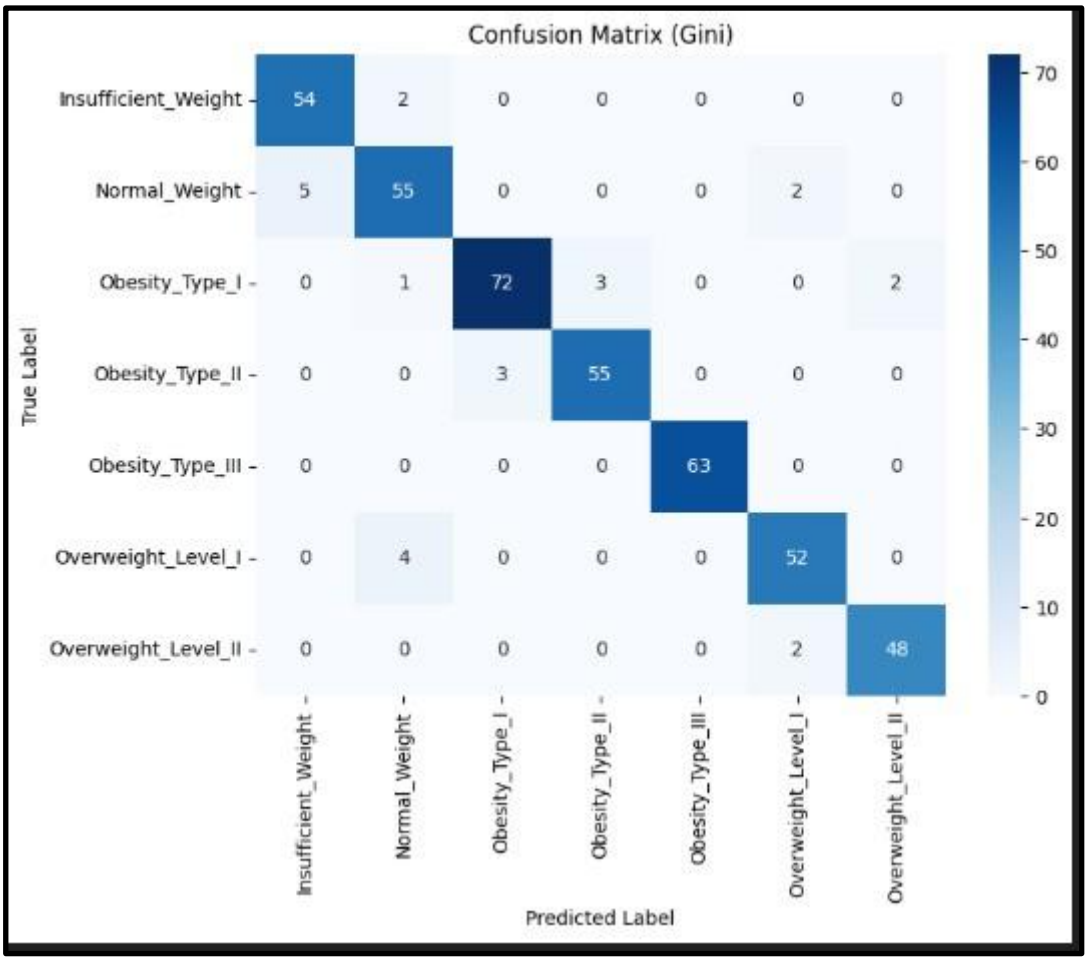
OUTPUT:

```
Accuracy (Gini, Full Tree): 0.9432624113475178

Confusion Matrix (Gini, Full Tree):

[[54 2 0 0 0 0 0]

[ 5 55 0 0 0 2 0]

[ 0 1 72 3 0 0 2]

[ 0 0 3 55 0 0 0]

[ 0 0 0 0 63 0 0]

[ 0 4 0 0 0 52 0]

[ 0 0 0 0 0 2 48]]
```

INPUT: Plot Confusion Matrix

```
# Compute confusion matrix for Gini

cm = confusion_matrix(y_test, y_pred)

# Plot confusion matrix (Gini)

plt.figure(figsize=(8,6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

xticklabels=target_names, yticklabels=target_names)

plt.title('Confusion Matrix (Gini)')

plt.ylabel('True Label')

plt.xlabel('Predicted Label')

plt.show()
```

OUTPUT:



ENTROPY

INPUT: Train Decision Tree

```
# Train and evaluate Decision Tree with Entropy

model_entropy = DecisionTreeClassifier(criterion='entropy')

model_entropy.fit(X_train, y_train)

y_pred_entropy = model_entropy.predict(X_test)

print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))

print("\nConfusion Matrix (Entropy, Full Tree):\n",

confusion_matrix(y_test,  y_pred_entropy))
```

OUTPUT:

Accuracy (Entropy, Full Tree): 0.9598108747044918

Confusion Matrix (Entropy, Full Tree):

[[53 3 0 0 0 0 0]

[5 56 0 0 0 1 0]

[0 0 76 2 0 0 0]

[0 0 2 56 0 0 0]

[0 0 0 0 63 0 0]

[0 0 0 0 0 56 0]

[0 0 1 0 0 3 46]]

INPUT: Evaluate Model

```
# Compute confusion matrix for Entropy
print("Calculating confusion matrix (Entropy)...")
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
```

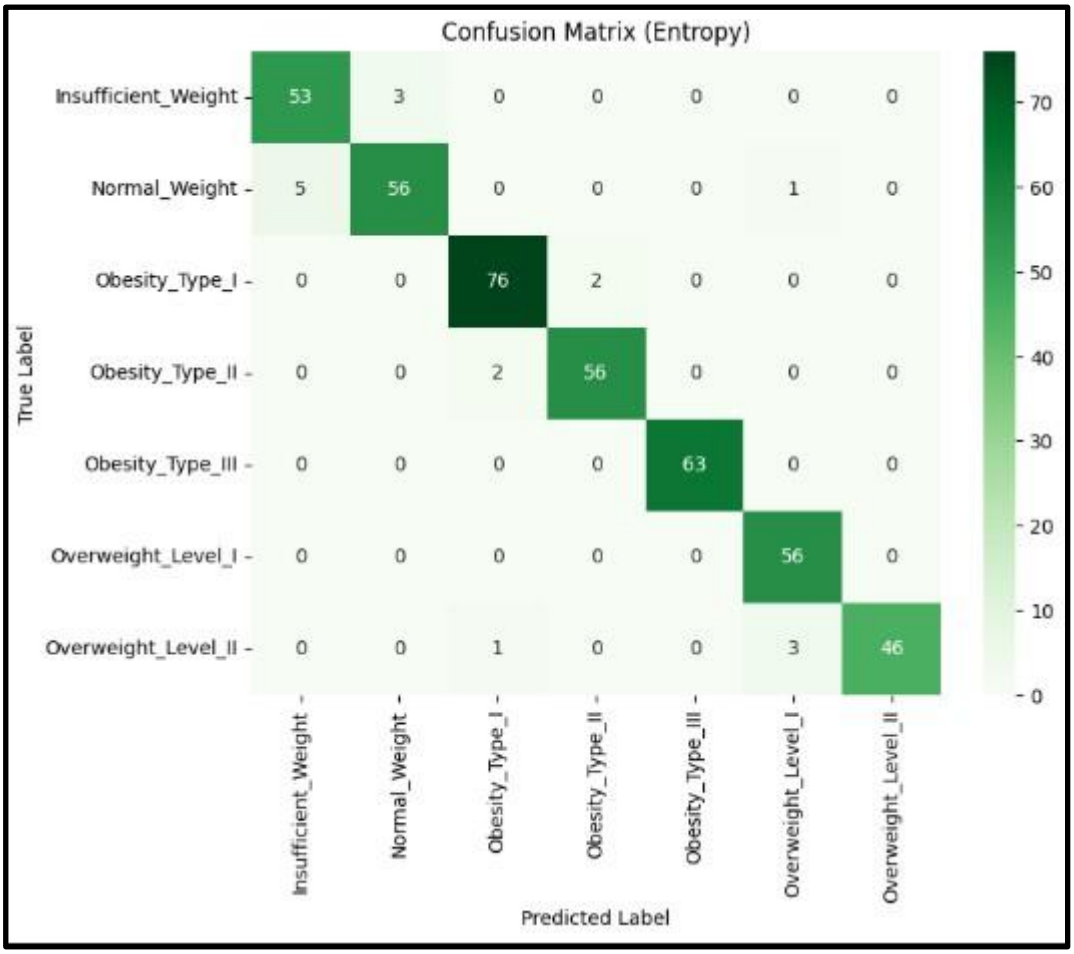
OUTPUT:

Calculating confusion matrix (Entropy)...

INPUT: Plot Confusion Matrix

```
# Plot confusion matrix (Entropy)
print("Displaying plot...")
plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens',
xticklabels=target_names, yticklabels=target_names)
plt.title('Confusion Matrix (Entropy)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```



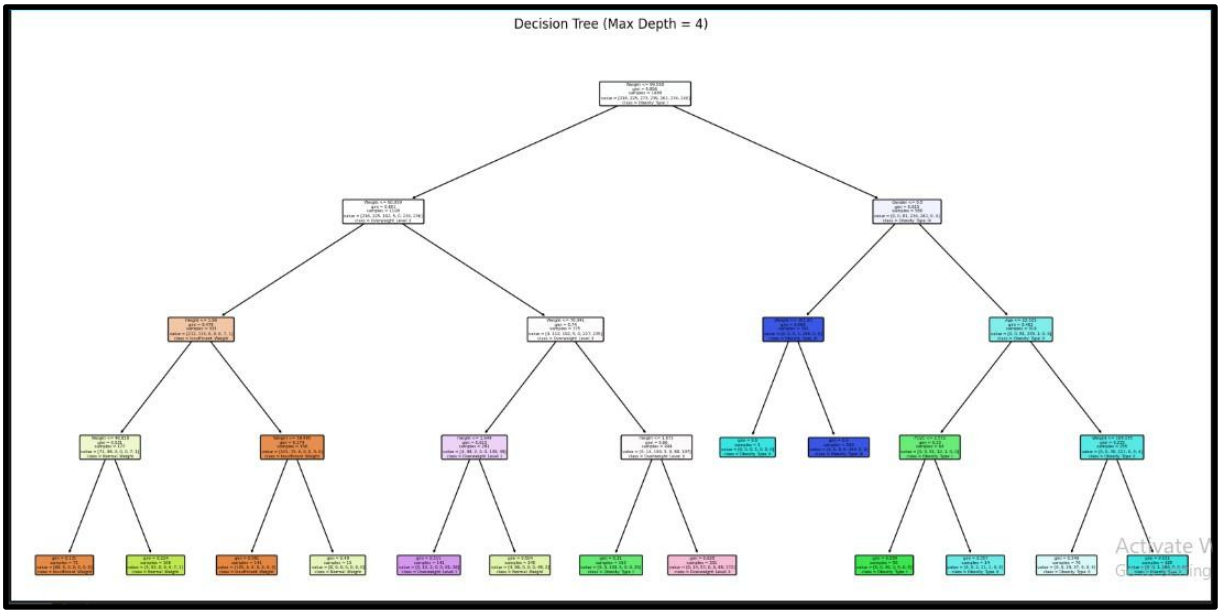
INPUT: GINI(MAX_DEPTH=4)

```
# Set maximum depth
d = 4
```

```
# Initialize Decision Tree with max depth = 4

clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)

# Train the model

clf.fit(X_train, y_train)

# Evaluate the model

acc = accuracy_score(y_test, clf.predict(X_test))

print(f"Max Depth: {d}, Accuracy: {acc:.4f}")

# Plot the Decision Tree

plt.figure(figsize=(20,10))

plot_tree(

clf,

feature_names=X.columns,

class_names=target_names,

filled=True,

rounded=True

)

plt.title(f"Decision Tree (Max Depth = {d})")

plt.show()
```

OUTPUT

```
Max Depth: 4, Accuracy: 0.7541
```



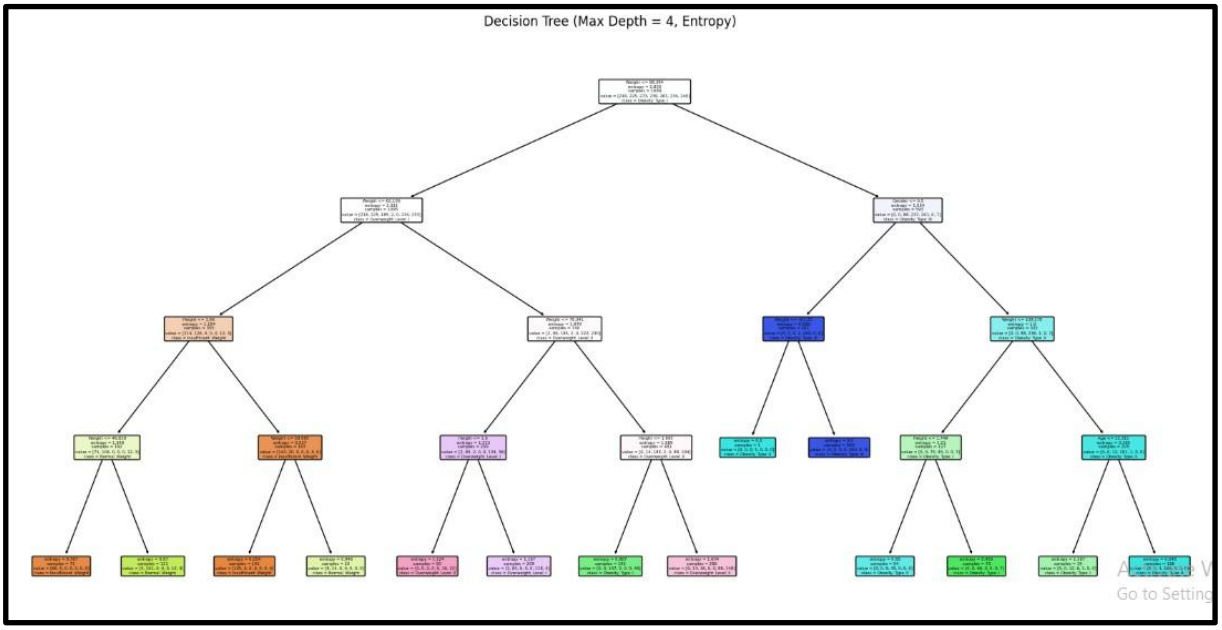
INPUT: ENTROPY(MAX_DEPTH=4)

```
# Analyze Decision Tree with Max Depth = 2 (Entropy)

d = 2

# Initialize Decision Tree with entropy criterion

clf = DecisionTreeClassifier(

max_depth=d,

criterion='entropy',

random_state=42

)

# Train the model

clf.fit(X_train, y_train)

# Evaluate the model

acc = accuracy_score(y_test, clf.predict(X_test))
```

```
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")

# Plot the Decision Tree

plt.figure(figsize=(20,10))

plot_tree(

clf,

feature_names=X.columns,

class_names=target_names,

filled=True,

rounded=True

)

plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")

plt.show()
```

OUTPUT:

```
Max Depth (Entropy): 4, Accuracy: 0.7400
```


Decision Tree (Max Depth = 4, Entropy)

Decision Trees - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Option A: Binary Classification (Loan Approval Dataset)

Step 1: Importing Libraries

 # Import necessary libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully!")
```

Output:

```
Libraries imported successfully!
```

Step 2: Initial Data Loading & Exploration

 # Load the dataset

```
data = pd.read_csv("loan_approval_dataset.csv")
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

 # Check the shape of the dataset

```
print("Dataset Shape:", data.shape)
```

Output:

```
Dataset Shape: (4269, 13)
```

 # Fix column spacing

```
data.columns = data.columns.str.strip()

# Print columns in vertical form
print("\\nColumns (Vertical List):")
for col in data.columns:
    print(col)
```

Output:

```
Columns (Vertical List):
loan_id
no_of_dependents
education
self_employed
income_annum
loan_amount
loan_term
cibil_score
residential_assets_value
commercial_assets_value
luxury_assets_value
bank_asset_value
loan_status
```

Display first 5 rows

```
print("First 5 rows of the dataset:")
data.head()
```

Output:

```
First 5 rows of the dataset:
loan_id  no_of_dependents  education     self_employed  income_annum  loan_amount  loan_term
cibil_score  residential_assets  commercial_assets_  luxury_assets_valu  bank_asset_value
loan_status
0  1        2                 Graduate      No             9600000       29900000     12
778          2400000             17600000            22700000            8000000
Approved
1  2        0                 Not Graduate  Yes            4100000       12200000     8
417          2700000             2200000             8800000             3300000
Rejected
2  3        3                 Graduate      No             9100000       29700000     20
506          7100000             4500000             33300000            12800000
Rejected
3  4        3                 Graduate      No             8200000       30700000     8
467          18200000            3300000             23300000            7900000
Rejected
4  5        5                 Not Graduate  Yes            9800000       24200000     20
382          12400000            8200000             29400000            5000000
Rejected
```

Display last 5 rows

```
print("Last 5 rows of the dataset:")
data.tail()
```

Output:

```
Last 5 rows of the dataset:
loan_id  no_of_dependents  education     self_employed  income_annum  loan_amount  loan_term
cibil_score  residential_assets  commercial_assets_  luxury_assets_valu  bank_asset_value
loan_status
4264  4265     5                 Graduate      Yes            1000000       2300000      12
317          2800000             500000              3300000             800000
Rejected
4265  4266     0                 Not Graduate  Yes            3300000       11300000     20
559          4200000             2900000             11000000            1900000
Approved
4266  4267     2                 Not Graduate  No             6500000       23900000     18
457          1200000             12400000            18100000            7300000
Rejected
4267  4268     1                 Not Graduate  No             4100000       12800000     8
780          8200000             700000              14100000            5800000
Approved
4268  4269     1                 Graduate      No             9200000       29700000     10
607          17800000            11800000            35700000            12000000
Approved
```

Step 3: Data Preprocessing & Information

Target Column Encoding

```
data.columns = data.columns.str.strip()
data['loan_status'] = data['loan_status'].str.strip().map({
    'Approved': 1,
    'Rejected': 0
})

print("Target column loan_status encoded successfully")
data.head()
```

Output:

```
Target column loan_status encoded successfully
loan_id  no_of_dependents  education     self_employed  income_annum  loan_amount  loan_term
cibil_score  residential_assets  commercial_assets_  luxury_assets_valu  bank_asset_value
loan_status
0  1        2                 Graduate      No             9600000       29900000     12
778          2400000             17600000            22700000            8000000           1
1  2        0                 Not Graduate  Yes            4100000       12200000     8
417          2700000             2200000             8800000             3300000           0
2  3        3                 Graduate      No             9100000       29700000     20
506          7100000             4500000             33300000            12800000          0
3  4        3                 Graduate      No             8200000       30700000     8
467          18200000            3300000             23300000            7900000           0
4  5        5                 Not Graduate  Yes            9800000       24200000     20
382          12400000            8200000             29400000            5000000           0
```

Data Information

```
print("All Required Information Related To Data: ")
data.info()
```

Output:

```
All Required Information Related To Data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4269 entries, 0 to 4268
Data columns (total 13 columns):
#   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
0   loan_id                   4269 non-null   int64
1   no_of_dependents          4269 non-null   int64
2   education                 4269 non-null   object
3   self_employed             4269 non-null   object
4   income_annum              4269 non-null   int64
5   loan_amount               4269 non-null   int64
6   loan_term                 4269 non-null   int64
7   cibil_score               4269 non-null   int64
8   residential_assets_value  4269 non-null   int64
9   commercial_assets_value   4269 non-null   int64
10  luxury_assets_value       4269 non-null   int64
11  bank_asset_value          4269 non-null   int64
12  loan_status               4269 non-null   int64
dtypes: int64(11), object(2)
memory usage: 433.7+ KB
```

 # Check for missing (null) values in each column

```
print("Missing values in each column:")
print(data.isnull().sum())
```

Output:

```
Missing values in each column:
loan_id                     0
no_of_dependents            0
education                   0
self_employed               0
income_annum                0
loan_amount                 0
loan_term                   0
cibil_score                 0
residential_assets_value    0
commercial_assets_value     0
luxury_assets_value         0
bank_asset_value            0
loan_status                 0
dtype: int64
```

Step 4: Feature Engineering & Data Splitting

 # Define target column

```
target_col = "loan_status"
print("Target column selected:", target_col)

# Separate features and target
X = data.drop(columns=[target_col])
y = data[target_col]
print("Features (X) and target (y) separated successfully")

# Identify categorical columns
cat_cols = X.select_dtypes(include=['object']).columns.tolist()
print("Categorical columns identified:", cat_cols)

# Apply One-Hot Encoding to categorical columns
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ],
    remainder='passthrough'
)

# Transform feature data
X_transformed = preprocessor.fit_transform(X)
print("Categorical features one-hot encoded and dataset transformed successfully")
```

Output:

```
Target column selected: loan_status
Features (X) and target (y) separated successfully
Categorical columns identified: ['education', 'self_employed']
Categorical features one-hot encoded and dataset transformed successfully
```

Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X_transformed,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Dataset successfully split into training and testing sets")
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

Output:

```
Dataset successfully split into training and testing sets
Training set size: (3415, 14)
Testing set size: (854, 14)
```

Step 5: Model Training & Evaluation (Gini Criterion)

Initialize and train Decision Tree model using Gini impurity

```
dt_model = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_model.fit(X_train, y_train)
print("Decision Tree model trained successfully using Gini impurity")
```

Output:

```
Decision Tree model trained successfully using Gini impurity
```

Evaluate Decision Tree model (Gini impurity)

```
y_pred = dt_model.predict(X_test)
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
print("\\nConfusion Matrix (Gini, Full Tree):")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
Accuracy (Gini, Full Tree): 0.9789227166276346
Confusion Matrix (Gini, Full Tree):
[[313  10]
[  8 523]]
```

Compute confusion matrix for Decision Tree model (Gini impurity)

```
print("Calculating confusion matrix (Gini)...")
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix calculated successfully")
```

Output:

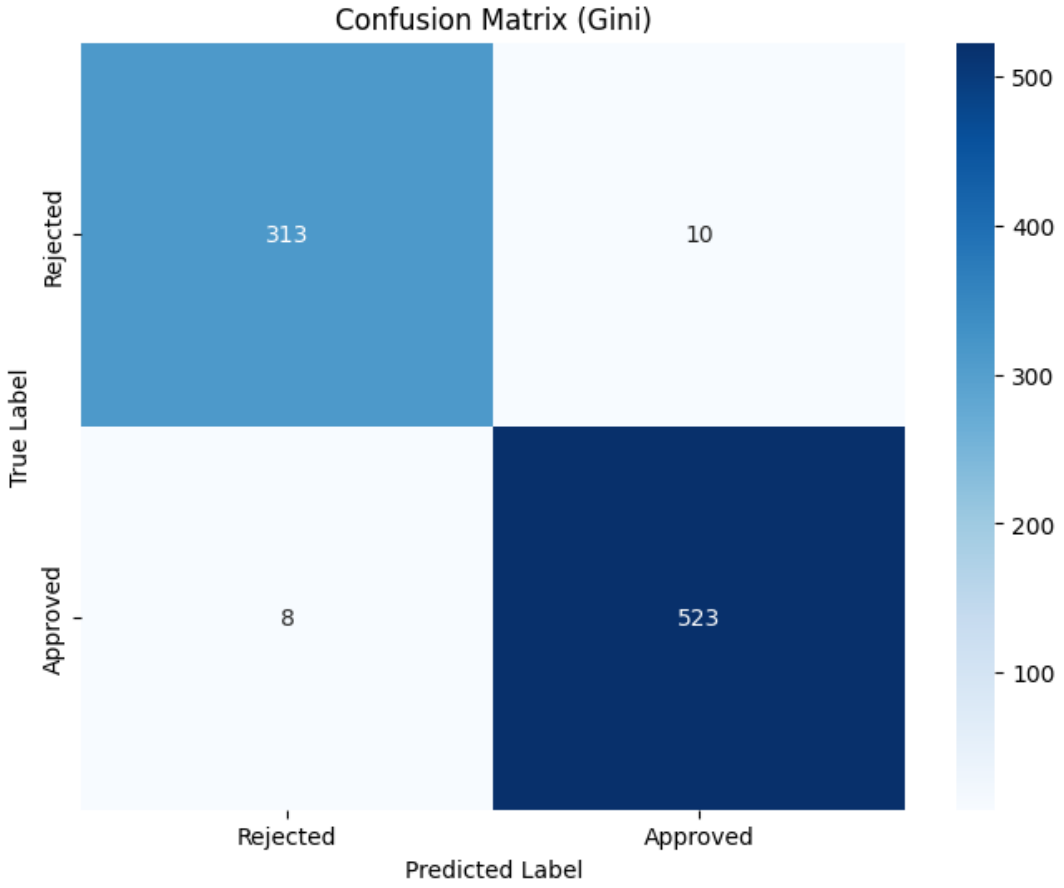
```
Calculating confusion matrix (Gini)...
Confusion matrix calculated successfully
```

Plot confusion matrix for Decision Tree model (Gini impurity)

```
print("Displaying confusion matrix plot (Gini)...")
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Rejected', 'Approved'],
    yticklabels=['Rejected', 'Approved'])
plt.title('Confusion Matrix (Gini)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
print("Confusion matrix plot displayed successfully")
```

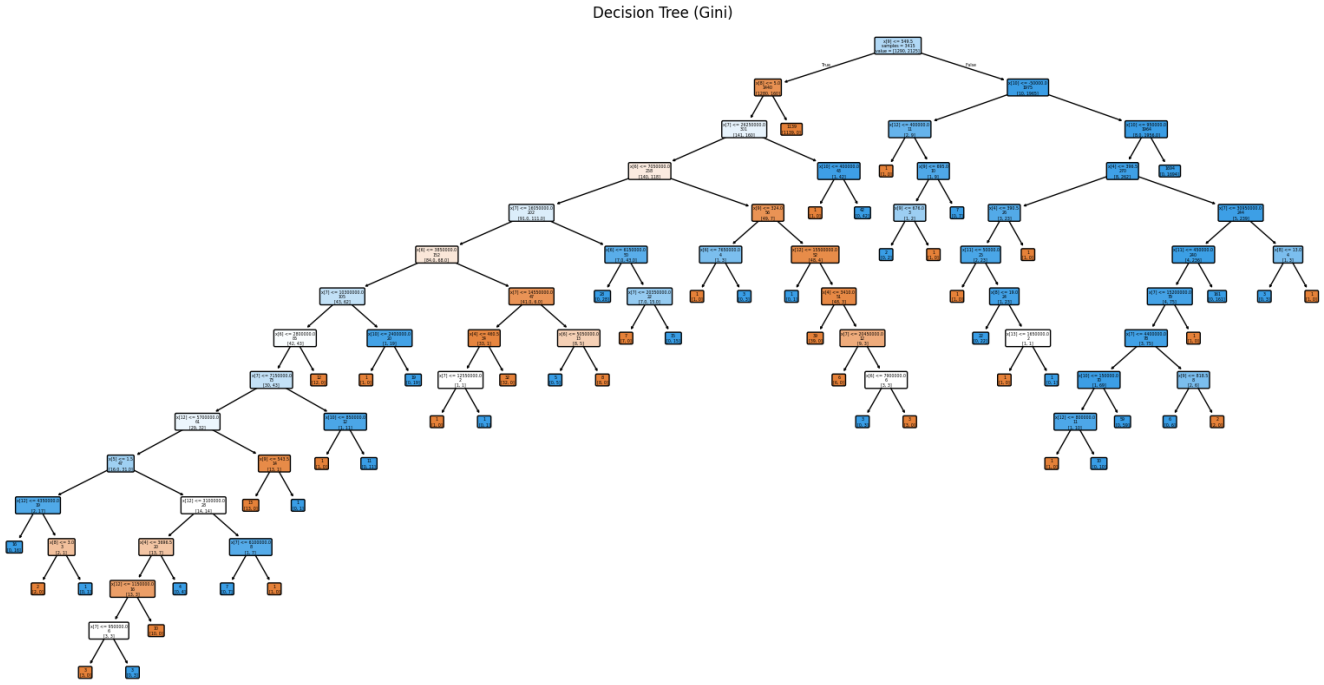
Output:

```
Displaying confusion matrix plot (Gini)...Confusion matrix plot displayed successfully
```



 # Visualize the Gini Decision Tree

```
plt.figure(figsize=(20, 10))
plot_tree(dt_model, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Gini)")
plt.show()
```



Step 6: Model Training & Evaluation (Entropy Criterion)

 # Train and evaluate Decision Tree model using Entropy

```
dt_model_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_model_entropy.fit(X_train, y_train)
print("Decision Tree model trained successfully using Entropy")

# Predict on test data
y_pred_entropy = dt_model_entropy.predict(X_test)
print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))
print("\\nConfusion Matrix (Entropy, Full Tree):")
print(confusion_matrix(y_test, y_pred_entropy))
```

Output:

```
Decision Tree model trained successfully using EntropyAccuracy (Entropy, Full Tree):
0.9789227166276346
Confusion Matrix (Entropy, Full Tree):
[[312  11]
[  7 524]]
```

 # Compute confusion matrix for Decision Tree model (Entropy)

```
print("Calculating confusion matrix (Entropy)...")
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
print("Confusion matrix (Entropy) calculated successfully")
```

Output:

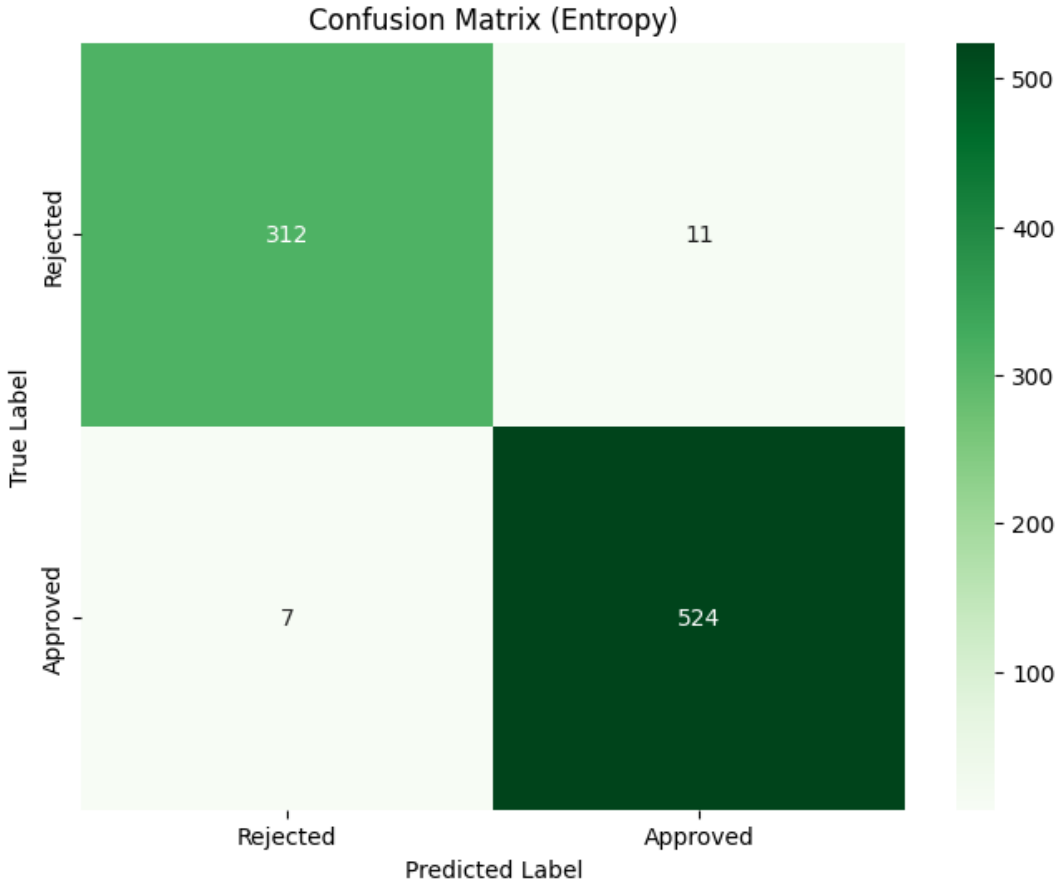
```
Calculating confusion matrix (Entropy)...
Confusion matrix (Entropy) calculated successfully
```

 # Plot confusion matrix for Decision Tree model (Entropy)

```
target_names = ['Rejected', 'Approved']
plt.figure(figsize=(8, 6))
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,
    yticklabels=target_names)
plt.title('Confusion Matrix (Entropy)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
print("Confusion matrix (Entropy) plot displayed successfully")
```

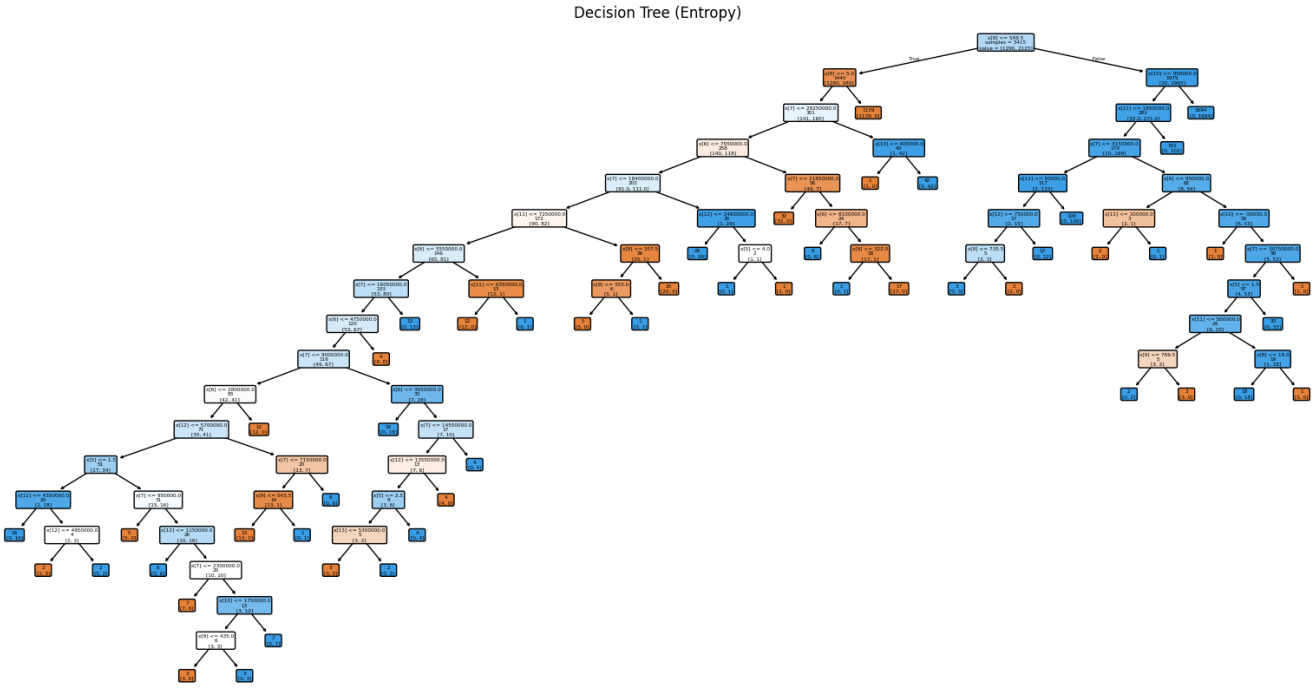
Output:

```
Displaying confusion matrix plot (Entropy)...Confusion matrix (Entropy) plot displayed
successfully
```



Visualize the Entropy Decision Tree

```
plt.figure(figsize=(20, 10))
plot_tree(dt_model_entropy, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Entropy)")
plt.show()
```



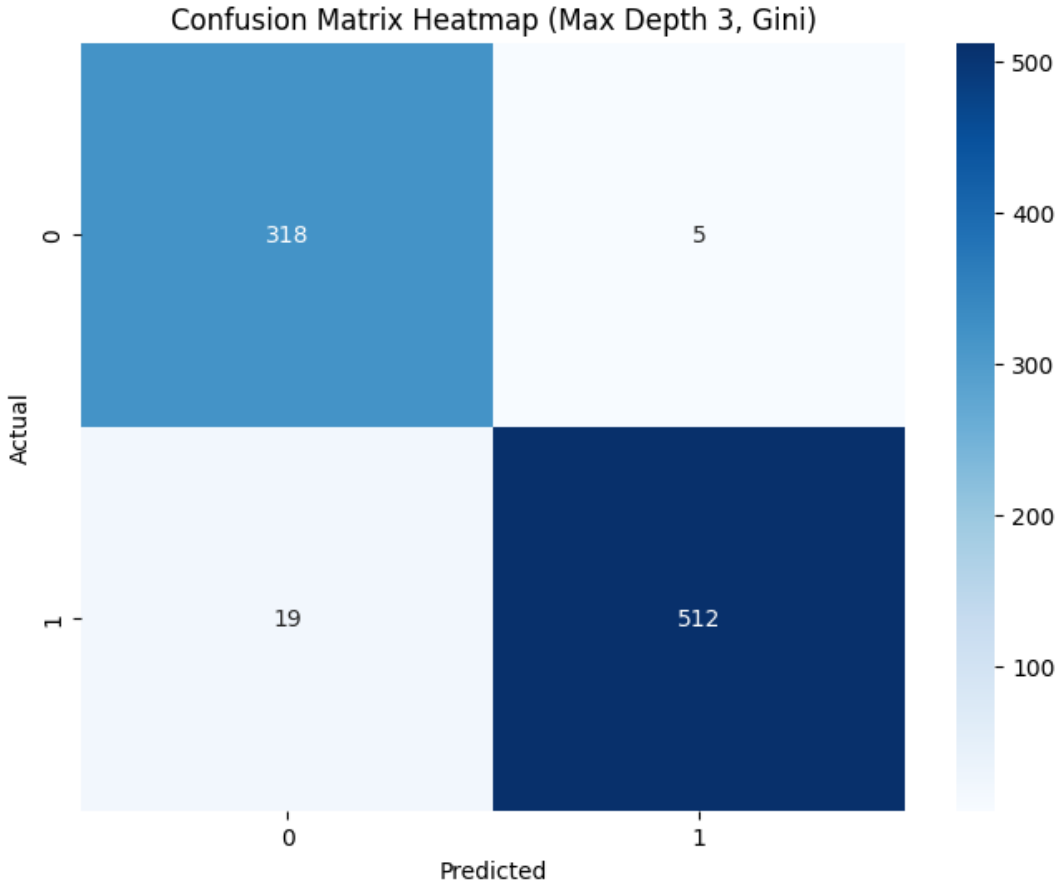
Step 7: Decision Tree Depth Analysis (Gini)

 # Train Decision Tree with max_depth=3 (Gini)

```
dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt_gini_3.fit(X_train, y_train)
y_pred_3 = dt_gini_3.predict(X_test)
accuracy_3 = accuracy_score(y_test, y_pred_3)
print(f"Accuracy (Max Depth 3, Gini): {accuracy_3:.4f}")
print("Confusion Matrix (Max Depth 3):")
print(confusion_matrix(y_test, y_pred_3))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_3), annot=True, fmt='d', cmap='Blues',
    xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 3, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

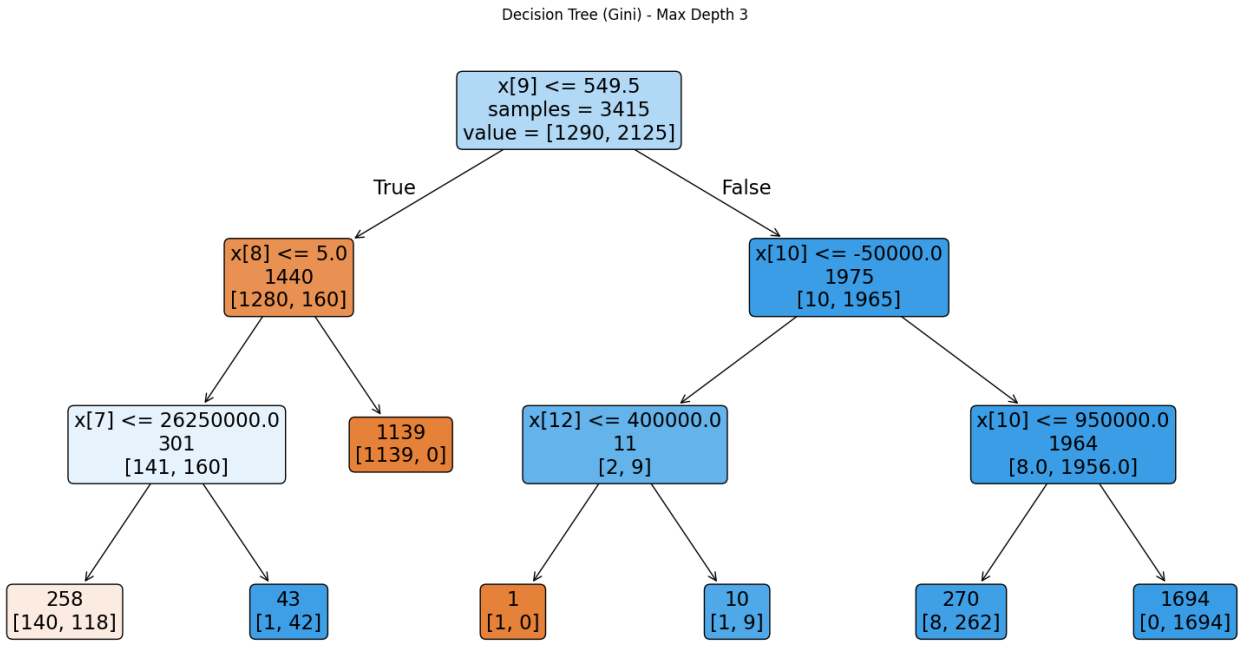
Output:

```
Accuracy (Max Depth 3, Gini): 0.9403
Confusion Matrix (Max Depth 3):
[[280  43]
[  8 523]]
```



 # Plot the tree (Gini - Max Depth 3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_3, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 3")
plt.show()
```

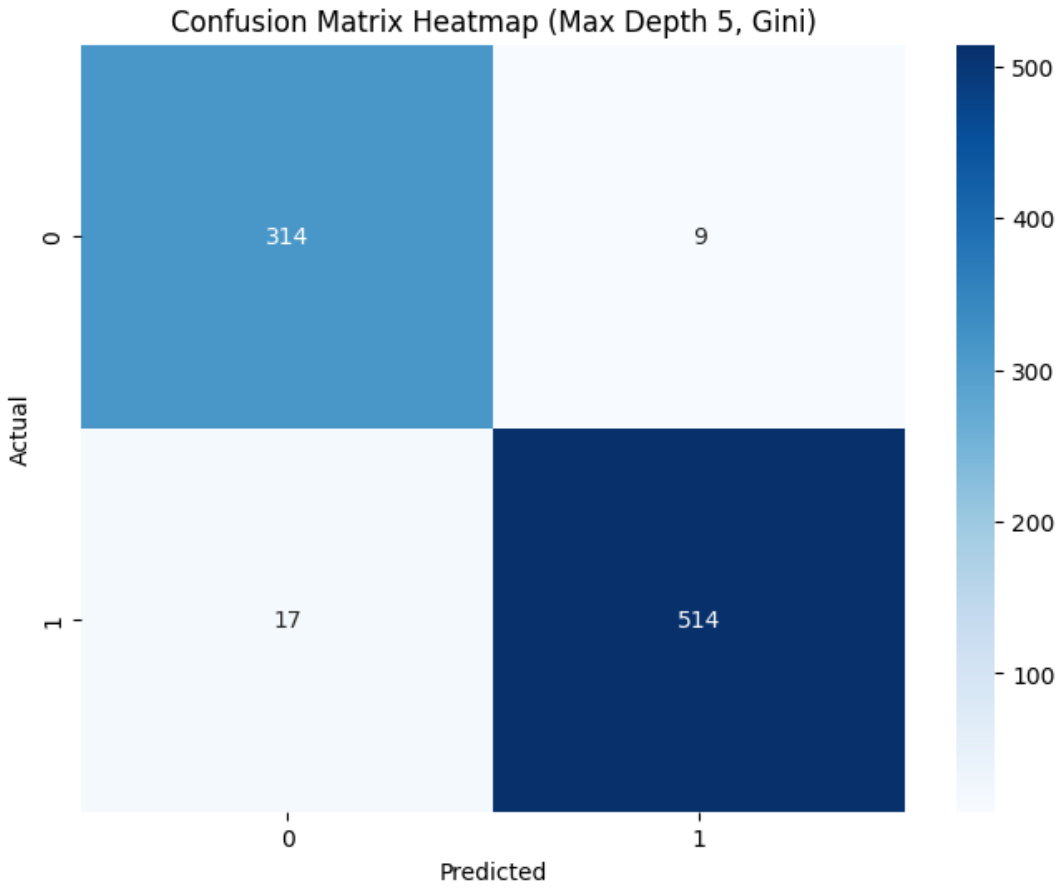


Train Decision Tree with max_depth=5 (Gini)

```
dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)
dt_gini_5.fit(X_train, y_train)
y_pred_5 = dt_gini_5.predict(X_test)
accuracy_5 = accuracy_score(y_test, y_pred_5)
print(f"Accuracy (Max Depth 5, Gini): {accuracy_5:.4f}")
print("Confusion Matrix (Max Depth 5):")
print(confusion_matrix(y_test, y_pred_5))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_5), annot=True, fmt='d', cmap='Blues',
    xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 5, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

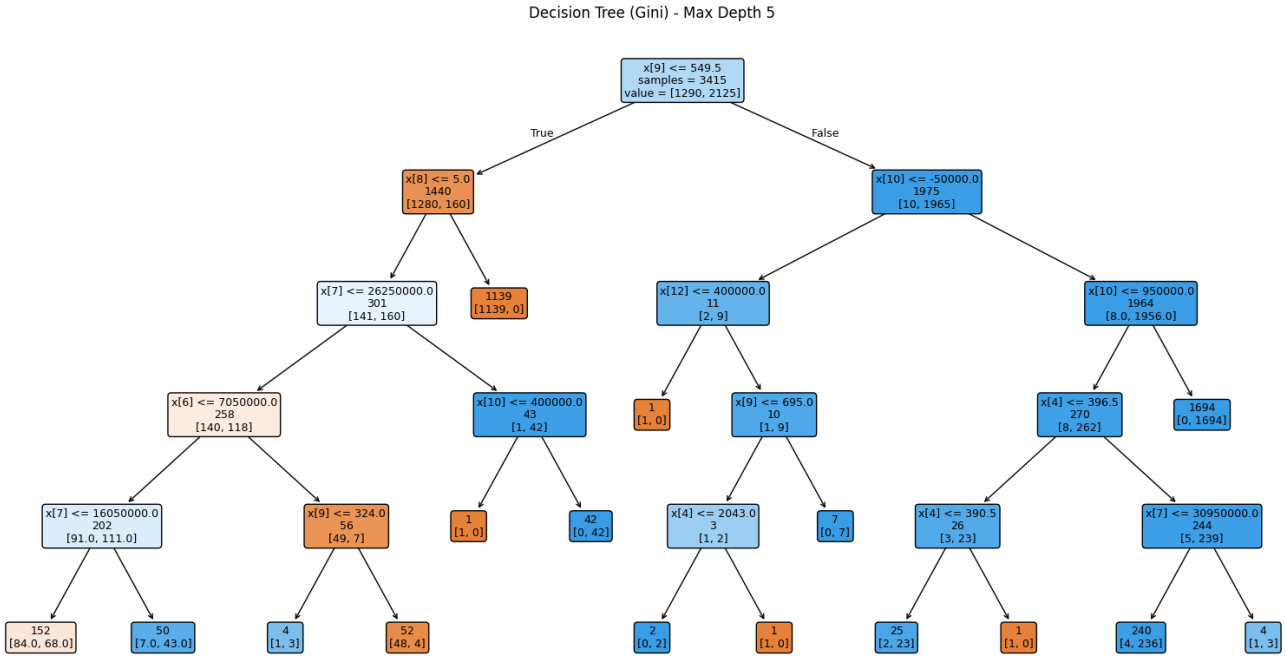
Output:

```
Accuracy (Max Depth 5, Gini): 0.9707
Confusion Matrix (Max Depth 5):
[[308  15]
[ 10 521]]
```



 # Plot the tree (Gini - Max Depth 5)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_5, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 5")
plt.show()
```

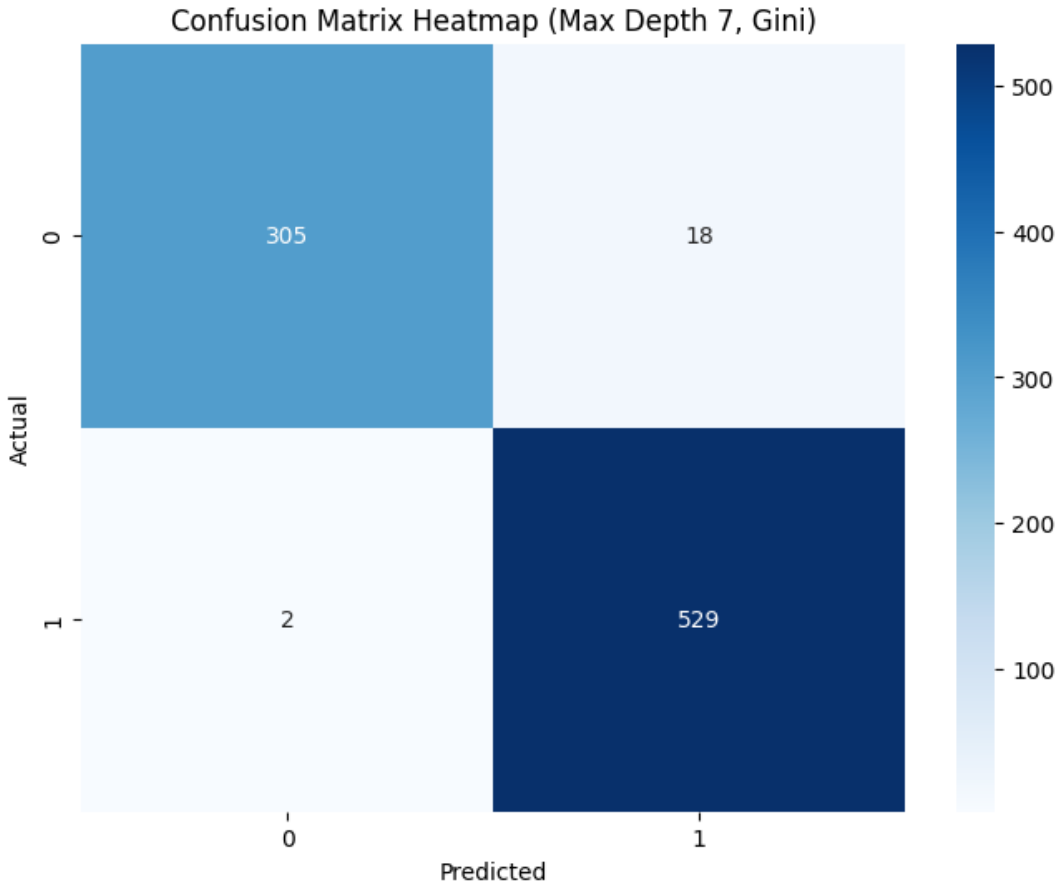


Train Decision Tree with max_depth=7 (Gini)

```
dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)
dt_gini_7.fit(X_train, y_train)
y_pred_7 = dt_gini_7.predict(X_test)
accuracy_7 = accuracy_score(y_test, y_pred_7)
print(f"Accuracy (Max Depth 7, Gini): {accuracy_7:.4f}")
print("Confusion Matrix (Max Depth 7):")
print(confusion_matrix(y_test, y_pred_7))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_7), annot=True, fmt='d', cmap='Blues',
    xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 7, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

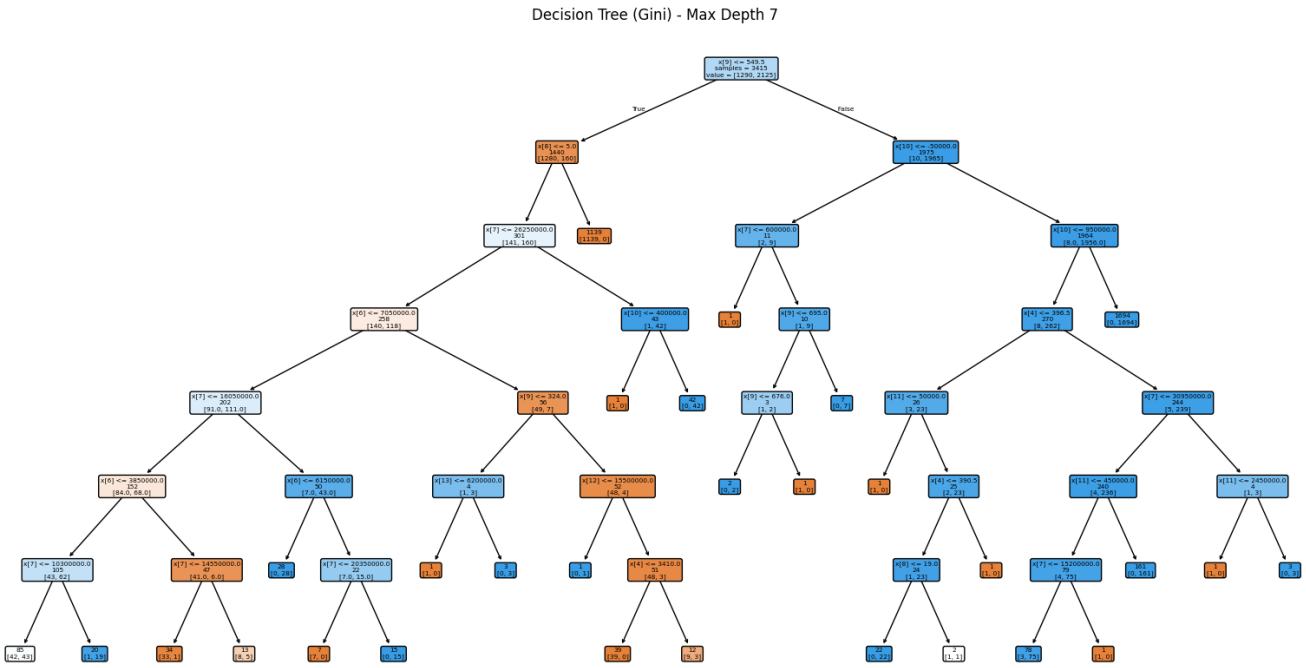
Output:

```
Accuracy (Max Depth 7, Gini): 0.9766
Confusion Matrix (Max Depth 7):
[[312  11]
[  9 522]]
```



Plot the tree (Gini - Max Depth 7)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_7, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 7")
plt.show()
```



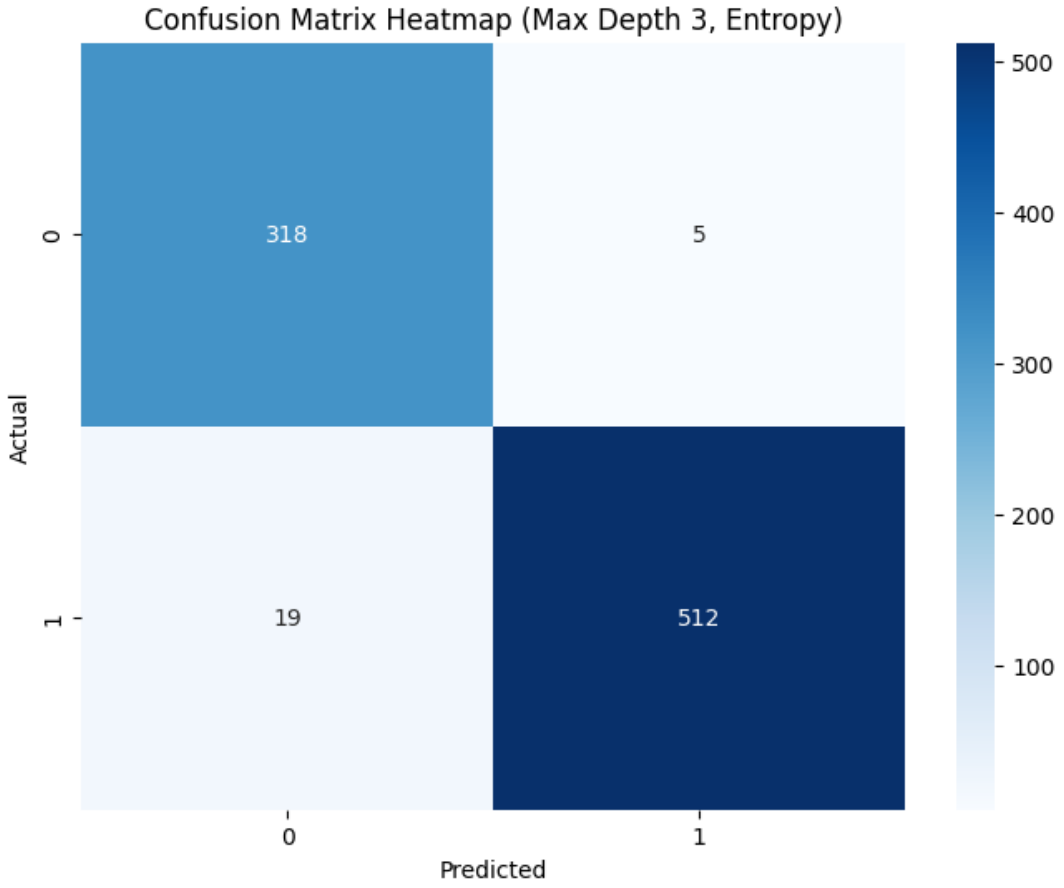
Step 8: Decision Tree Depth Analysis (Entropy)

 # Train Decision Tree with max_depth=3 (Entropy)

```
dt_entropy_3 = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
dt_entropy_3.fit(X_train, y_train)
y_pred_3 = dt_entropy_3.predict(X_test)
accuracy_3 = accuracy_score(y_test, y_pred_3)
print(f"Accuracy (Max Depth 3, Entropy): {accuracy_3:.4f}")
print("Confusion Matrix (Max Depth 3):")
print(confusion_matrix(y_test, y_pred_3))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_3), annot=True, fmt='d', cmap='Blues',
    xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 3, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

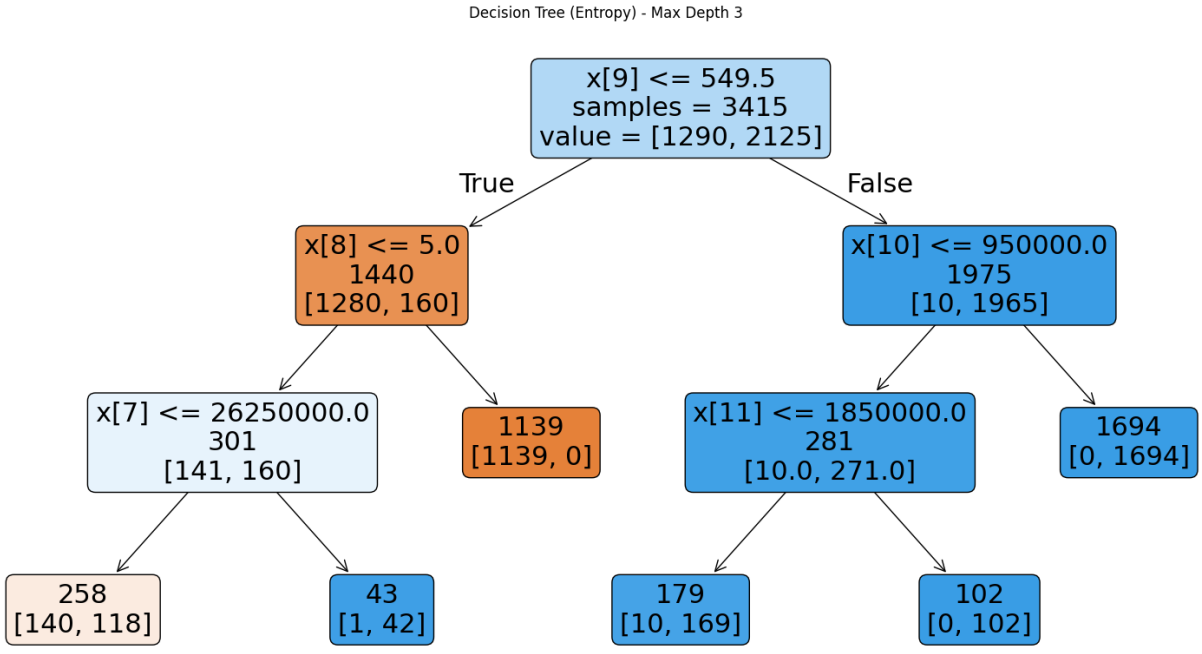
Output:

```
Accuracy (Max Depth 3, Entropy): 0.9333
Confusion Matrix (Max Depth 3):
[[275  48]
[  9 522]]
```



 # Plot the tree (Entropy - Max Depth 3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_3, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 3")
plt.show()
```

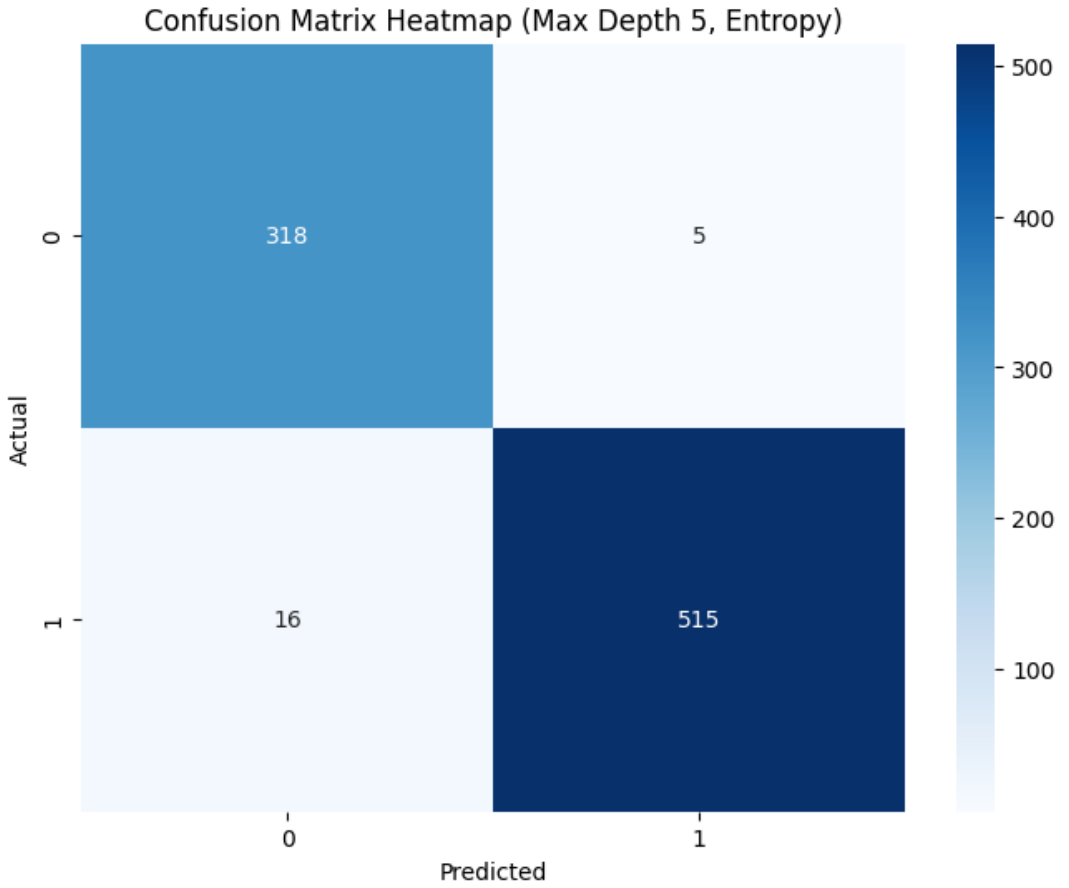


Train Decision Tree with max_depth=5 (Entropy)

```
dt_entropy_5 = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=42)
dt_entropy_5.fit(X_train, y_train)
y_pred_5 = dt_entropy_5.predict(X_test)
accuracy_5 = accuracy_score(y_test, y_pred_5)
print(f"Accuracy (Max Depth 5, Entropy): {accuracy_5:.4f}")
print("Confusion Matrix (Max Depth 5):")
print(confusion_matrix(y_test, y_pred_5))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_5), annot=True, fmt='d', cmap='Blues',
    xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 5, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

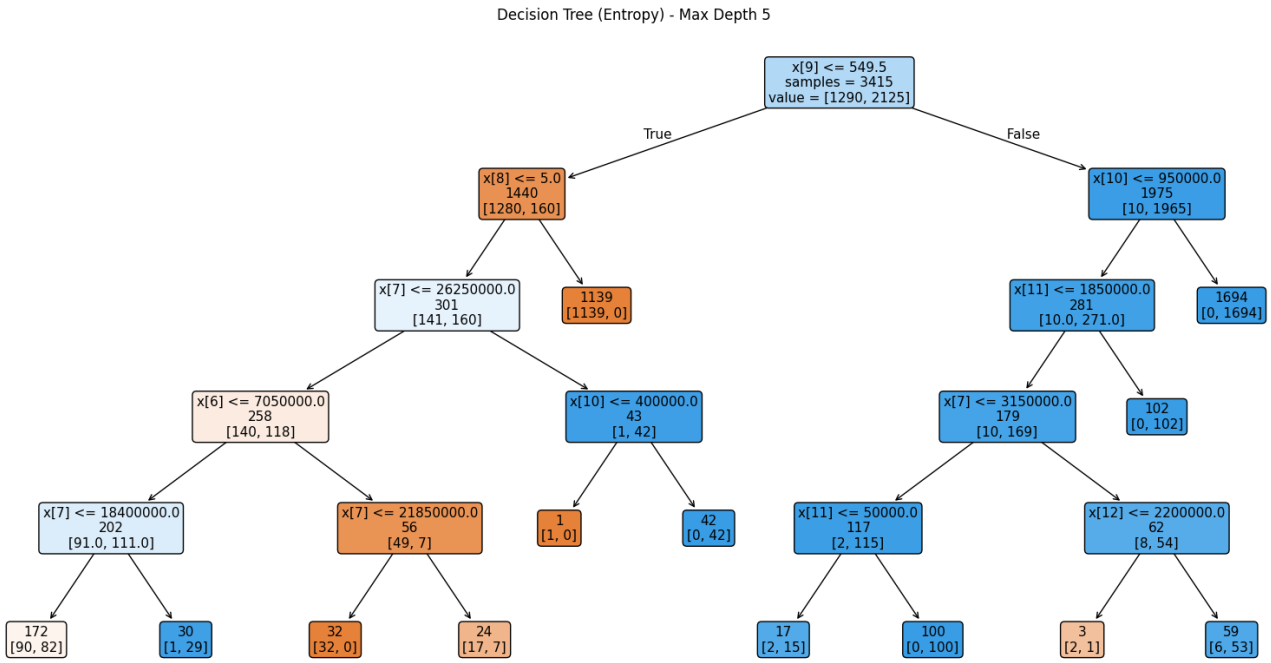
Output:

```
Accuracy (Max Depth 5, Entropy): 0.9637
Confusion Matrix (Max Depth 5):
[[304  19]
[ 12 519]]
```



Plot the tree (Entropy - Max Depth 5)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_5, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 5")
plt.show()
```

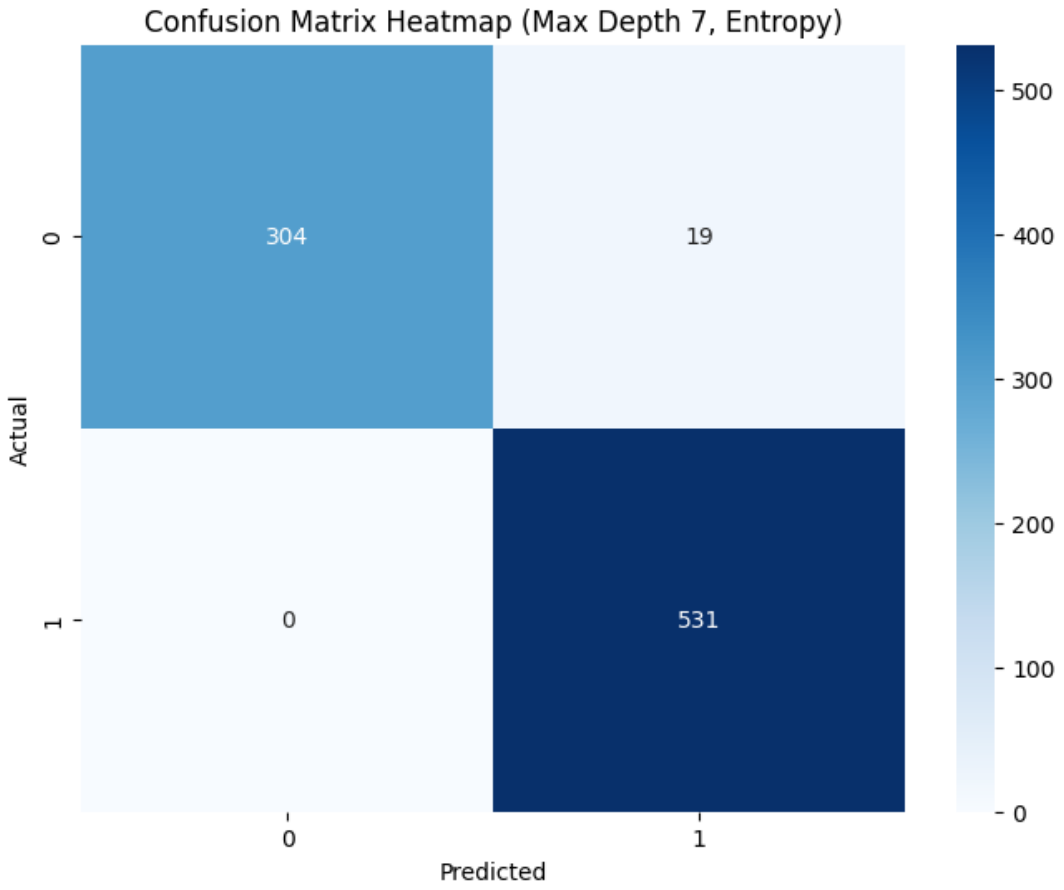


Train Decision Tree with max_depth=7 (Entropy)

```
dt_entropy_7 = DecisionTreeClassifier(criterion='entropy', max_depth=7, random_state=42)
dt_entropy_7.fit(X_train, y_train)
y_pred_7 = dt_entropy_7.predict(X_test)
accuracy_7 = accuracy_score(y_test, y_pred_7)
print(f"Accuracy (Max Depth 7, Entropy): {accuracy_7:.4f}")
print("Confusion Matrix (Max Depth 7):")
print(confusion_matrix(y_test, y_pred_7))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_7), annot=True, fmt='d', cmap='Blues',
    xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 7, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

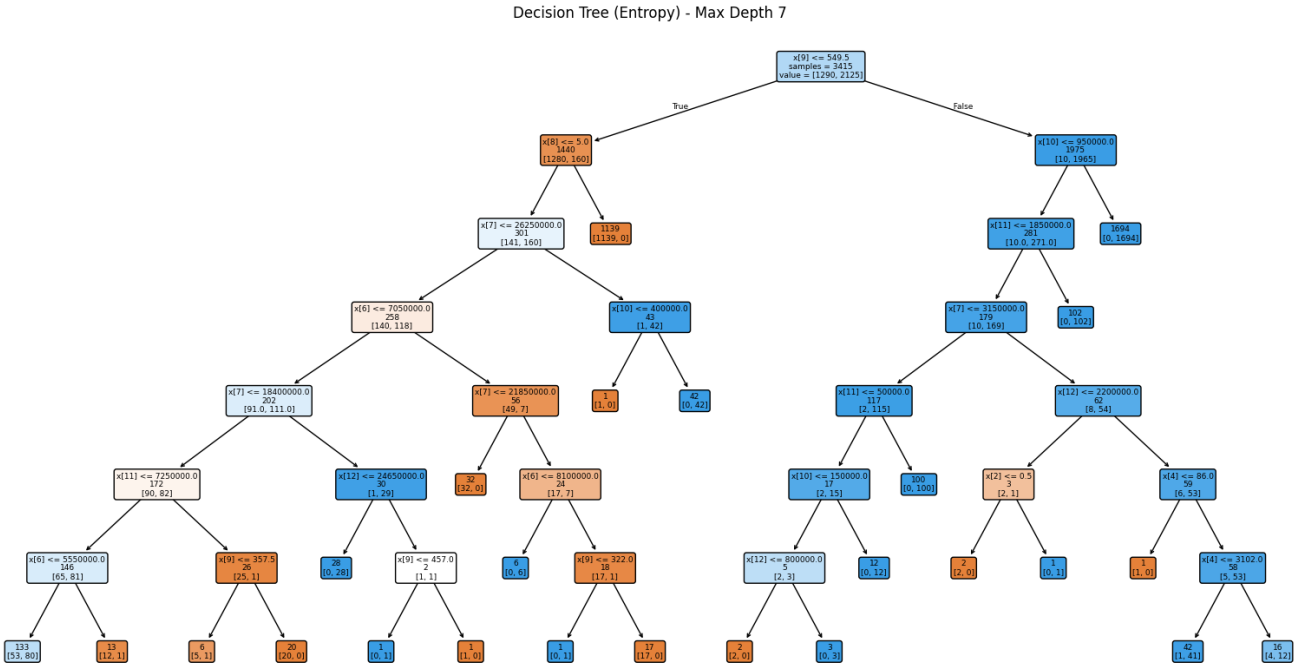
Output:

```
Accuracy (Max Depth 7, Entropy): 0.9766
Confusion Matrix (Max Depth 7):
[[312  11]
[  9 522]]
```



Plot the tree (Entropy - Max Depth 7)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_7, filled=True, rounded=True, label='root', impurity=False,
    proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 7")
plt.show()
```



Option B: Multiclass Classification (Obesity Dataset)

Step 1: Importing Libraries

 # Import necessary libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
print("Libraries imported successfully!")
```

Output:

```
Libraries imported successfully!
```

Step 2: Loading Data

 # Load the dataset

```
data = pd.read_csv('ObesityDataSet_raw_and_data_sinthetic.csv')
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

 # Check the shape of the dataset

```
print("Data Shape:")
print(data.shape)
```

Output:

```
Data Shape:
(2111, 17)
```

 # Display first 5 rows of the dataset

```
print("First 5 rows of the dataset:")
data.head()
```

Output:

```
First 5 rows of the dataset:
Gender  Age   Height  Weight  family_history_wit  FAVC  FCVC  NCP  CAEC       SMOKE  CH2O  SCC
FAF  TUE  CALC        MTRANS              NObeyesdad
0  Female  21.0  1.62    64.0    yes                 no    2.0   3.0  Sometimes  no     2.0
no   0.0  1.0  no          Public_Transportat  Normal_Weight
1  Female  21.0  1.52    56.0    yes                 no    3.0   3.0  Sometimes  yes    3.0
yes  3.0  0.0  Sometimes   Public_Transportat  Normal_Weight
2  Male    23.0  1.80    77.0    yes                 no    2.0   3.0  Sometimes  no     2.0
no   2.0  1.0  Frequently  Public_Transportat  Normal_Weight
3  Male    27.0  1.80    87.0    no                  no    3.0   3.0  Sometimes  no     2.0
no   2.0  0.0  Frequently  Walking             Overweight_Level_I
4  Male    22.0  1.78    89.8    no                  no    2.0   1.0  Sometimes  no     2.0
no   0.0  0.0  Sometimes   Public_Transportat  Overweight_Level_I
```

 # Display last 5 rows of the dataset

```
print("Last 5 rows of the dataset:")
data.tail()
```

Output:

```
Last 5 rows of the dataset:
Gender  Age        Height    Weight      family_history_wit  FAVC  FCVC  NCP  CAEC       SMOKE
CH2O      SCC  FAF       TUE       CALC       MTRANS              NObeyesdad
2106  Female  20.976842  1.710730  131.408528  yes                 yes   3.0   3.0  Sometimes
no     1.728139  no   1.676269  0.906247  Sometimes  Public_Transportat  Obesity_Type_III
2107  Female  21.982942  1.748584  133.742943  yes                 yes   3.0   3.0  Sometimes
no     2.005130  no   1.341390  0.599270  Sometimes  Public_Transportat  Obesity_Type_III
2108  Female  22.524036  1.752206  133.689352  yes                 yes   3.0   3.0  Sometimes
no     2.054193  no   1.414209  0.646288  Sometimes  Public_Transportat  Obesity_Type_III
2109  Female  24.361936  1.739450  133.346641  yes                 yes   3.0   3.0  Sometimes
no     2.852339  no   1.139107  0.586035  Sometimes  Public_Transportat  Obesity_Type_III
2110  Female  23.664709  1.738836  133.472641  yes                 yes   3.0   3.0  Sometimes
no     2.863513  no   1.026452  0.714137  Sometimes  Public_Transportat  Obesity_Type_III
```

Step 3: Preprocessing

```
print("Statistical summary of the dataset:")
data.describe()
```

Output:

```
Statistical summary of the dataset:
Age          Height       Weight       FCVC         NCP          CH2O         FAF          TUE
count  2111.000000  2111.000000  2111.000000  2111.000000  2111.000000  2111.000000
2111.000000  2111.000000
mean    24.312600    1.701677     86.586058    2.419043     2.685628     2.008011     1.010298
0.657866
std     6.345968     0.093305     26.191172    0.533927     0.778039     0.612953     0.850592
0.608927
min     14.000000    1.450000     39.000000    1.000000     1.000000     1.000000     0.000000
0.000000
25%     19.947192    1.630000     65.473343    2.000000     2.658738     1.584812     0.124505
0.000000
50%     22.777890    1.700499     83.000000    2.385502     3.000000     2.000000     1.000000
0.625350
75%     26.000000    1.768464     107.430682   3.000000     3.000000     2.477420     1.666678
1.000000
max     61.000000    1.980000     173.000000   3.000000     4.000000     3.000000     3.000000
2.000000
```

Initialize LabelEncoder

```
le = LabelEncoder()
target_names = []
print("LabelEncoder initialized")
```

Output:

```
LabelEncoder initialized
```

Apply encoding to categorical columns

```
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])
        if col == 'NObeyesdad':
            target_names = list(le.classes_)
print("Categorical variables encoded")
```

Output:

```
Categorical variables encoded
```

Split data into training and testing sets

```
X = data.drop('NObeyesdad', axis=1)
y = data['NObeyesdad']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data split complete")
```

Output:

```
Data split complete
```

Step 4: Training (Gini)

 # Initialize and train Decision Tree with Gini impurity

```
model = DecisionTreeClassifier(criterion='gini')
model.fit(X_train, y_train)
print("Model trained using Gini impurity")
```

Output:

```
Model trained using Gini impurity
```

 # Evaluate Gini model

```
y_pred = model.predict(X_test)
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy (Gini, Full Tree): 0.9433
```

 # Compute confusion matrix for Gini

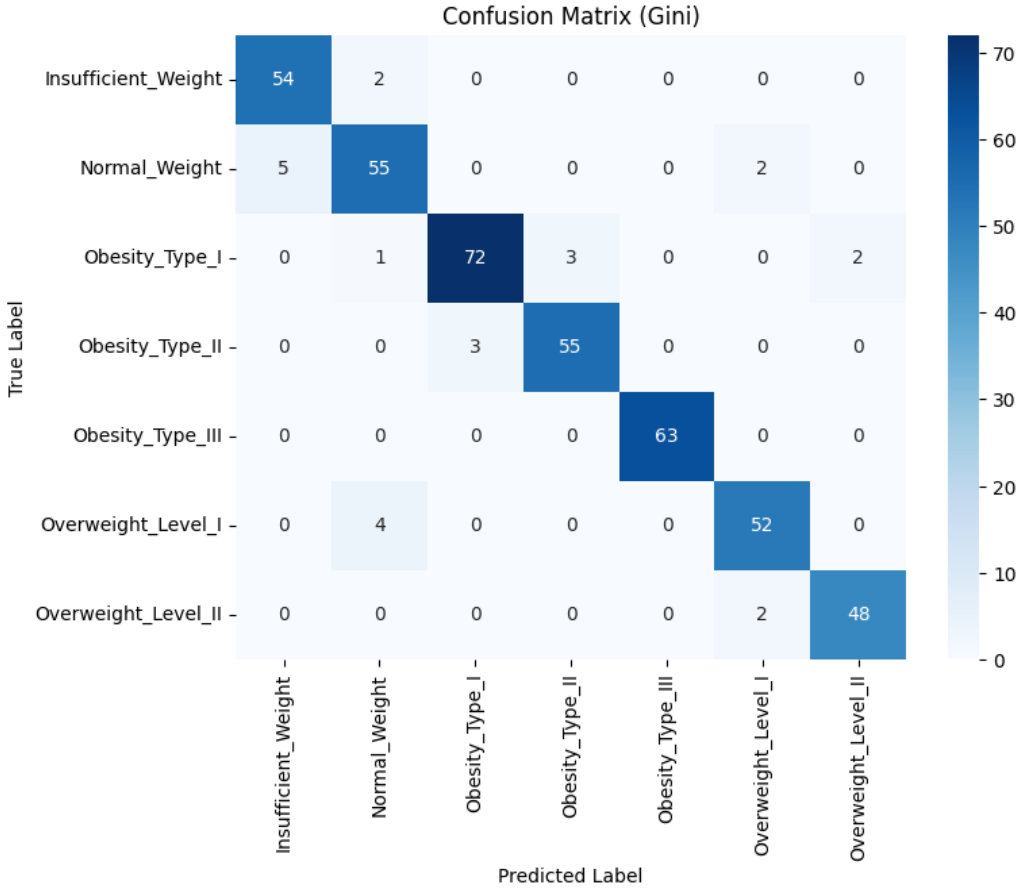
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix shape:", cm.shape)
```

Output:

```
Confusion matrix shape: (7, 7)
```

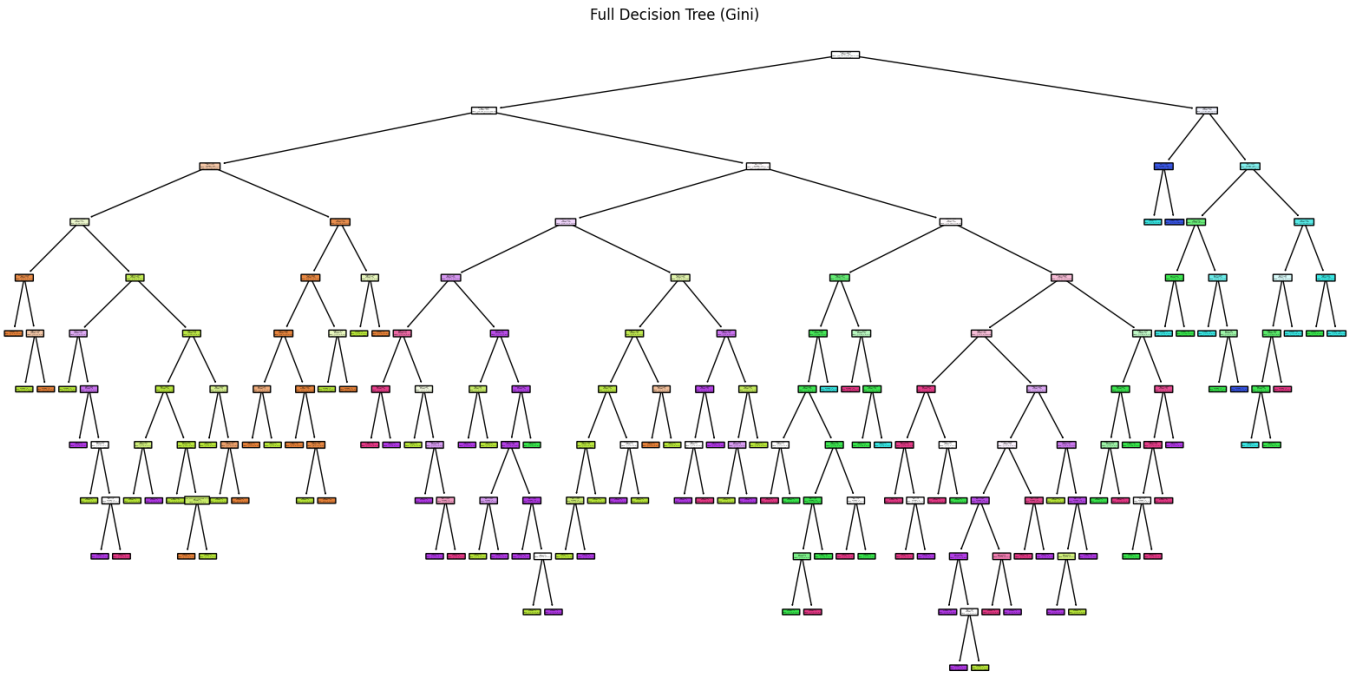
 # Plot confusion matrix (Gini)

```
print("Displaying plot...")
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=target_names,
    yticklabels=target_names)
plt.title('Confusion Matrix (Gini)')
plt.show()
```



Visualize the Gini Decision Tree

```
print("Plotting the Decision Tree (Gini)...")
plt.figure(figsize=(20,10))
plot_tree(model, feature_names=X.columns, class_names=target_names, filled=True,
    rounded=True)
plt.show()
```



Step 5: Training (Entropy)

Train and evaluate Decision Tree with Entropy

```
model_entropy = DecisionTreeClassifier(criterion='entropy')
model_entropy.fit(X_train, y_train)
y_pred_entropy = model_entropy.predict(X_test)
print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))
```

Output:

```
Accuracy (Entropy, Full Tree): 0.9598
```

Compute confusion matrix for Entropy

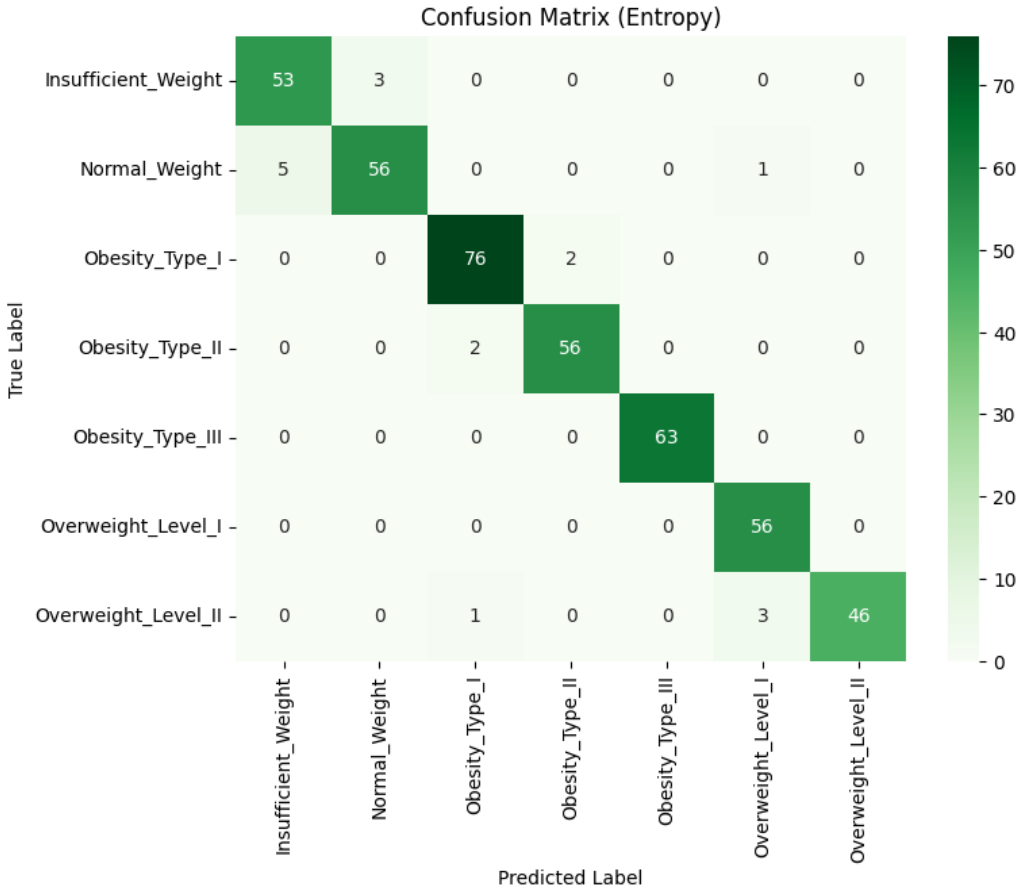
```
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
print("Confusion matrix shape:", cm_entropy.shape)
```

Output:

```
Confusion matrix shape: (7, 7)
```

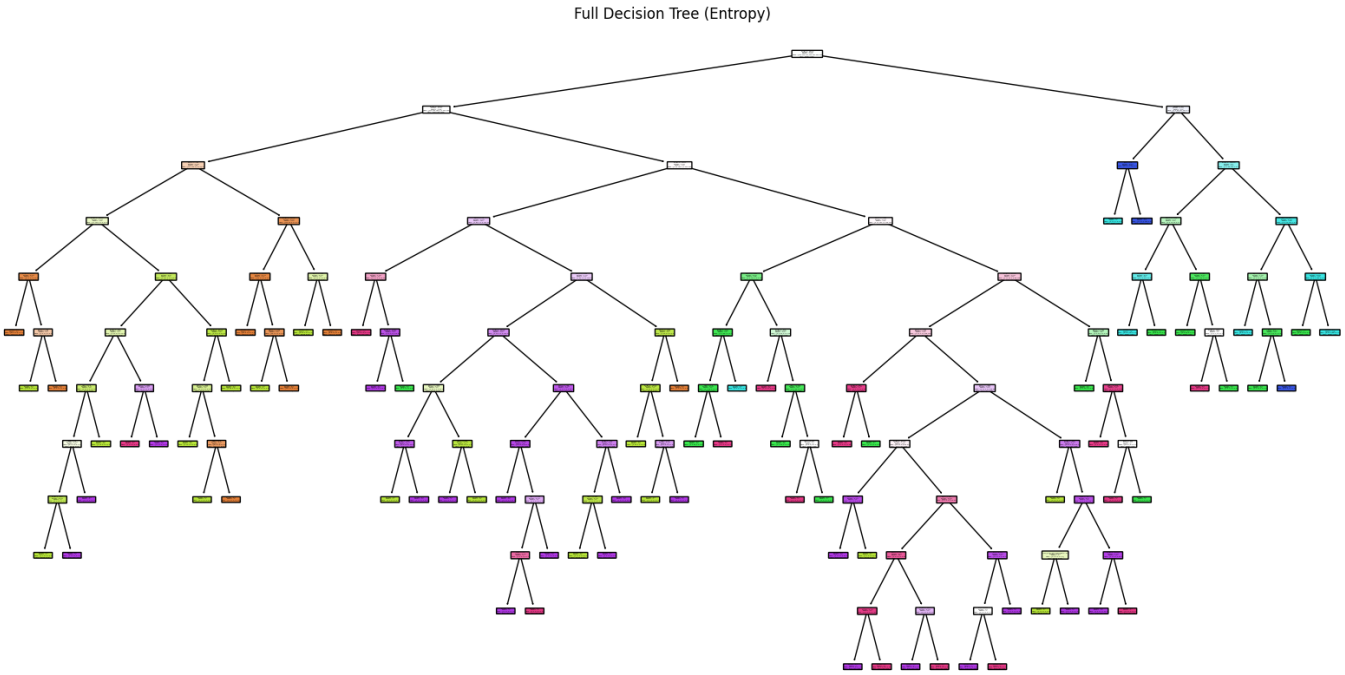
Plot confusion matrix (Entropy)

```
print("Displaying plot...")
plt.figure(figsize=(8,6))
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,
    yticklabels=target_names)
plt.show()
```



```
# Visualize the Entropy Decision Tree
```

```
print("Plotting the Decision Tree (Entropy)...")
plot_tree(model_entropy, feature_names=X.columns, class_names=target_names, filled=True,
    rounded=True)
plt.show()
```



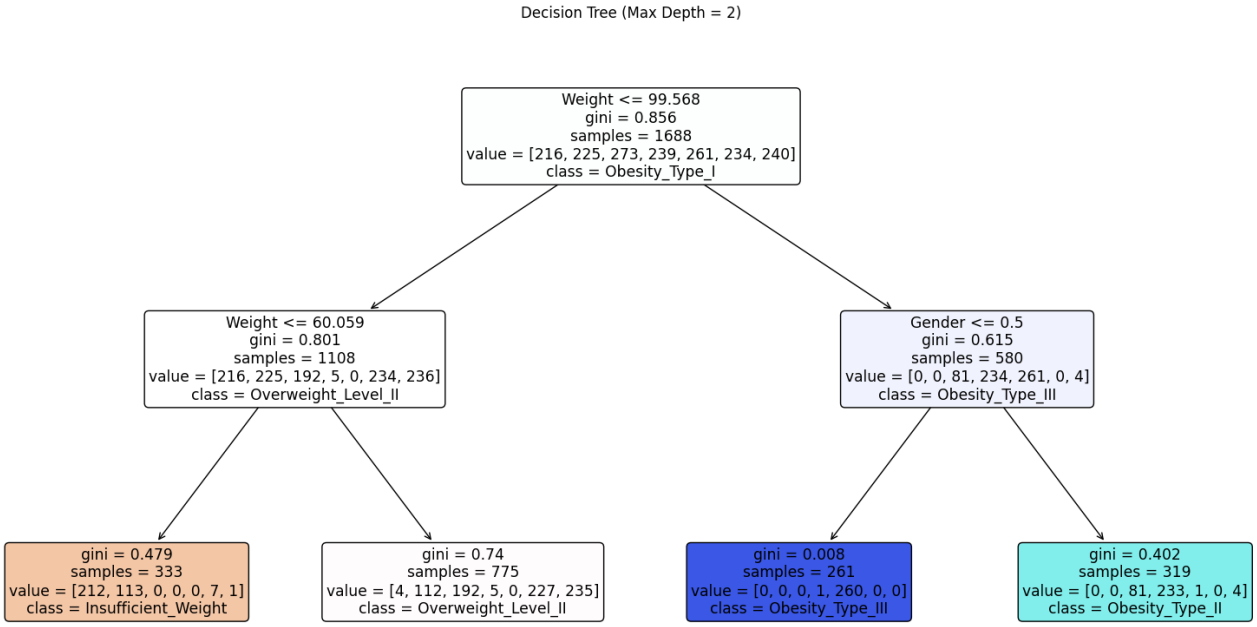
Step 6: Model Evaluation (Depth)

```
# Analyze Decision Tree with Max Depth = 2
```

```
d = 2
clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth: {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
    rounded=True)
plt.title(f"Decision Tree (Max Depth = {d})")
plt.show()
```

Output:

```
Max Depth: 2, Accuracy: 0.5432
```



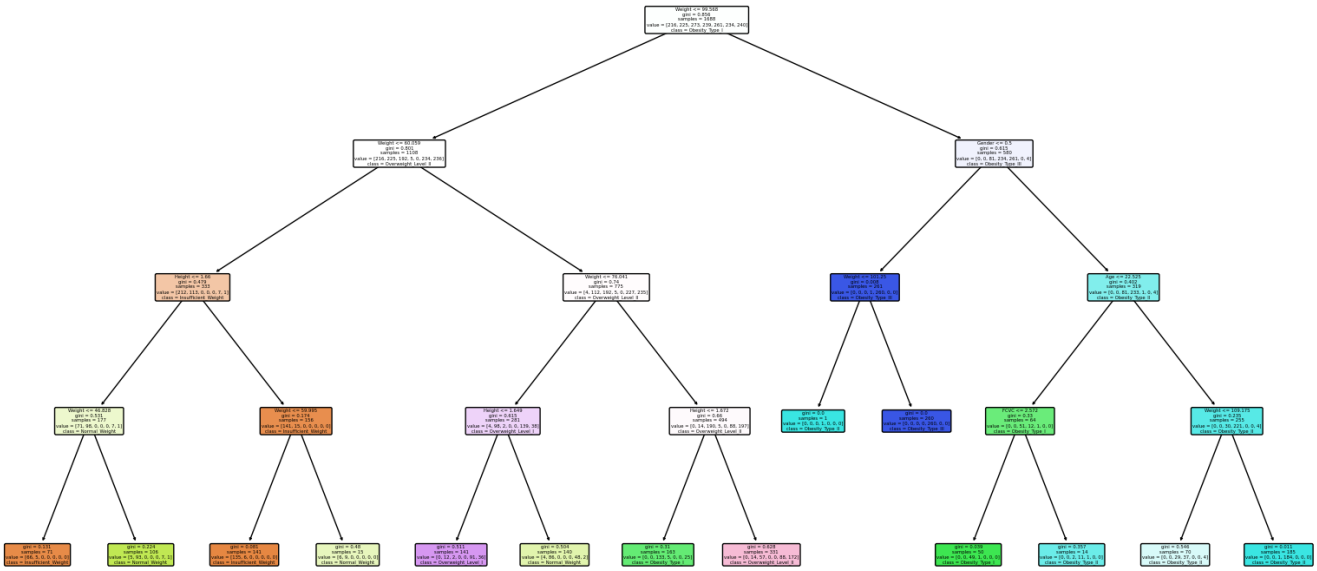
Analyze Decision Tree with Max Depth = 4

```
d = 4
clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth: {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
    rounded=True)
plt.title(f"Decision Tree (Max Depth = {d})")
plt.show()
```

Output:

```
Max Depth: 4, Accuracy: 0.9433
```

Decision Tree (Max Depth = 4)

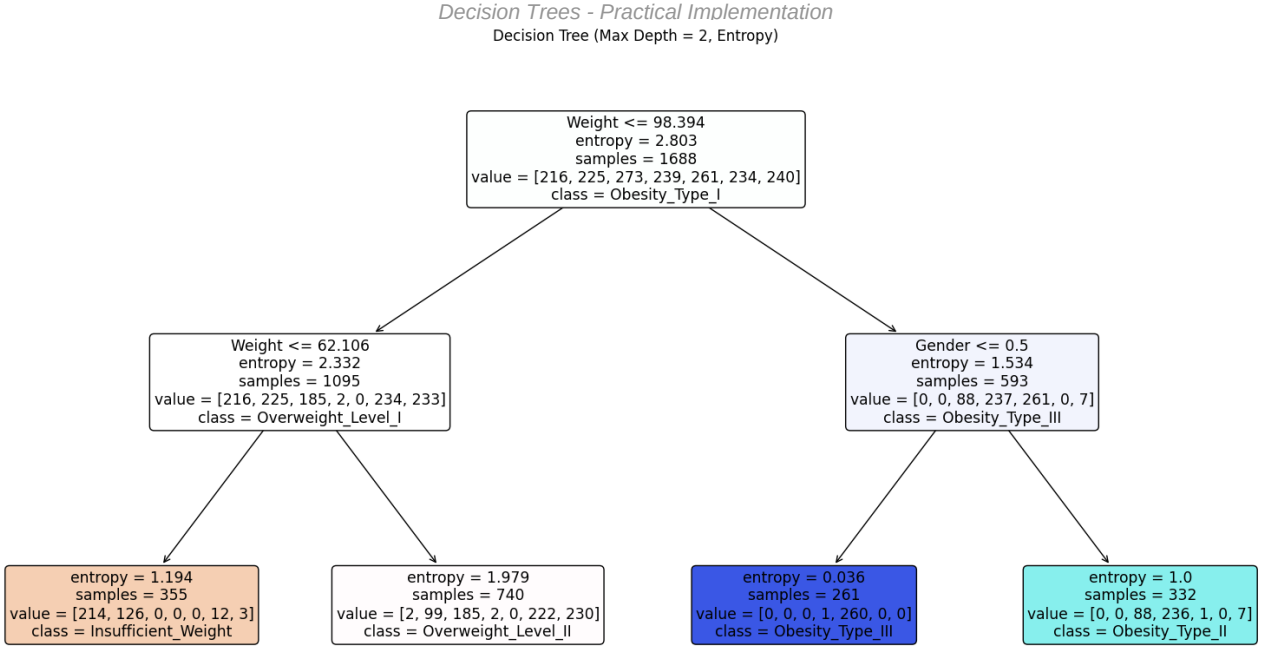


Analyze Decision Tree with Max Depth = 2 (Entropy)

```
d = 2
clf = DecisionTreeClassifier(max_depth=d, criterion='entropy', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
    rounded=True)
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
plt.show()
```

Output:

```
Max Depth (Entropy): 2, Accuracy: 0.9456
```



Analyze Decision Tree with Max Depth = 4 (Entropy)

```
d = 4
clf = DecisionTreeClassifier(max_depth=d, criterion='entropy', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
    rounded=True)
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
plt.show()
```

Output:

```
Max Depth (Entropy): 4, Accuracy: 0.9598
```

Decision Tree (Max Depth = 4, Entropy)